

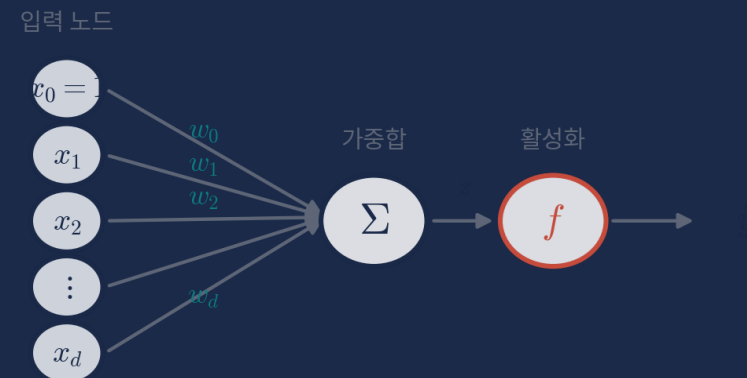
CHAPTER 5

# 퍼셉트론

구조와 동작 · 학습 규칙 · 논리 게이트 — 그리고 MLP, 트랜스포머, LLM으로

2026학년도 2학기 개정판

성명 : 최승호  
이메일 : jcn99250@naver.com



# 학습 목표

01

로지스틱 회귀와 퍼셉트론의 관계를 설명한다

시그모이드 활성화를 쓰는 퍼셉트론 = 로지스틱 회귀

02

퍼셉트론의 구조·순전파·학습 규칙을 수식으로 유도한다

가중합, 활성화, 오분류 기반 갱신, 경사 하강

03

수렴 정리와 마진의 의미를 이해하고 증명한다

Novikoff 상한  $(R/\gamma)^2$ , 암묵적 편향, VC 차원

04

논리 게이트로 선형 분리 가능성의 한계(XOR)를 설명한다

Minsky-Papert 1969 → 은닉층의 필요성

05

퍼셉트론에서 MLP·트랜스포머·LLM까지 계보를 잇는다

FFN = 2층 MLP, 어텐션 = 학습된 가중합

06

PyTorch로 퍼셉트론과 MLP를 구현하고 실험한다

sklearn → NumPy → PyTorch 실습 노트북

# 강의 로드맵 — 퍼셉트론에서 LLM까지



초록: 1943–2012 고전 신경망 · 주황: 2017– 트랜스포머·LLM 시대. 이번 강의는 1958년 퍼셉트론의 수식이 2026년 LLM 안에 그대로 살아 있음을 보이는 것이 목표입니다.

# A

## 퍼셉트론의 구조와 동작

로지스틱 회귀 복습 → 노드·에지·가중치 → 가중합과 활성화 → 순전파

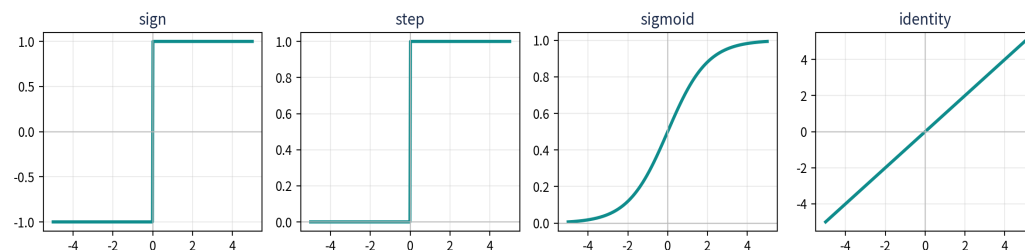


# 로지스틱 회귀 복습

- **이진 분류:** 모든 개체가 클래스  $C_1$  혹은  $C_2$ 에 속하는 상황
  - 접근 1:  $C_1$ 과  $C_2$ 를 구분하는 특징 공간의 분할(결정 경계)을 학습
  - 접근 2:  $P(C_1|x)$ 와  $P(C_2|x)$ 를 비교하여 레이블 결정
- **선형 판별 함수  $g(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$  — 특징 성분의 가중합**
- **로지스틱 회귀:** 사후 확률의 로그 오즈를 선형 함수로 근사
  - $\ln[P(C_1|x)/(1-P(C_1|x))] = \mathbf{w}^\top \mathbf{x} \Rightarrow P(C_1|x) = \sigma(\mathbf{w}^\top \mathbf{x})$

$$g(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}, \quad P(C_1 | \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}}$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z) (1 - \sigma(z))$$



오늘의 관점: 로지스틱 회귀는 "활성화 함수가 시그모이드인 퍼셉트론"이다 — 오른쪽 그림의 sign·step·sigmoid·identity 중 sigmoid를 고른 경우(슬라이드 11에서 상세).

# 퍼셉트론과 인공지능망

## 퍼셉트론 (Perceptron, Rosenblatt 1958)

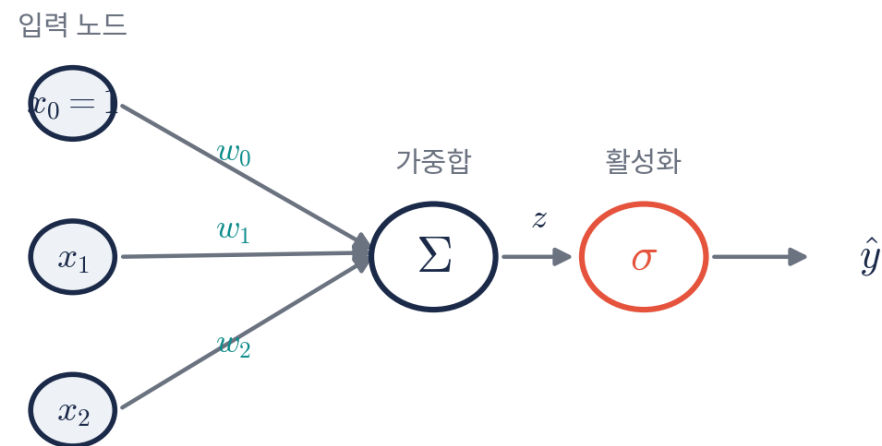
인공신경망의 기본 정보처리 유닛. 특징 벡터를 구성하는 성분의 가중합 연산에 비선형 활성화 함수를 적용하는 모델. McCulloch-Pitts(1943)의 논리 뉴런에 "학습 규칙"을 더한 것이 핵심 기여.

## 왜 다시 퍼셉트론인가 (2026)

트랜스포머 FFN의 뉴런 하나하나가 퍼셉트론이며, 어텐션은 학습된 가중합이다. 2018년 이후의 암묵적 편향(implicit bias) 이론은 퍼셉트론·로지스틱 회귀의 수렴 분석에서 출발한다.

## 학습 중점

- 퍼셉트론 모델의 구조를 그림으로 표현할 수 있는 능력 확보
- 퍼셉트론 구성 단위(노드·에지·가중치·활성화)의 역할 이해
- 퍼셉트론의 순전파와 학습 규칙의 수학적 이해
- 단일 퍼셉트론의 표현력 한계와 다층 구조로의 확장



# 퍼셉트론의 구조 (1/3) — 데이터의 표현

- 퍼셉트론 개요

- 다층 인공신경망의 기본 구성 단위로 주로 활용
- 특징 벡터의 성분에 대한 가중합 + 활성화 함수

- 데이터 표현

- d개의 성분으로 구성된 특징 벡터로 개별 관측값이 설명됨
- $\mathbf{x}_i = [x_{i1} \cdots x_{id}]^T$  : i번째 특징 벡터,  $x_{ij}$  : 그 j번째 성분
- $y_i$  : 관측 특징 벡터  $\mathbf{x}_i$ 에 대한 정답 / 목표값

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N, \quad \mathbf{x}_i = [x_{i1}, \dots, x_{id}]^T \in \mathbb{R}^d, \quad y_i \in \{0, 1\}$$

## 예: 필기체 숫자 (sklearn digits)

8×8 = 64 픽셀 → d = 64 차원 특징 벡터

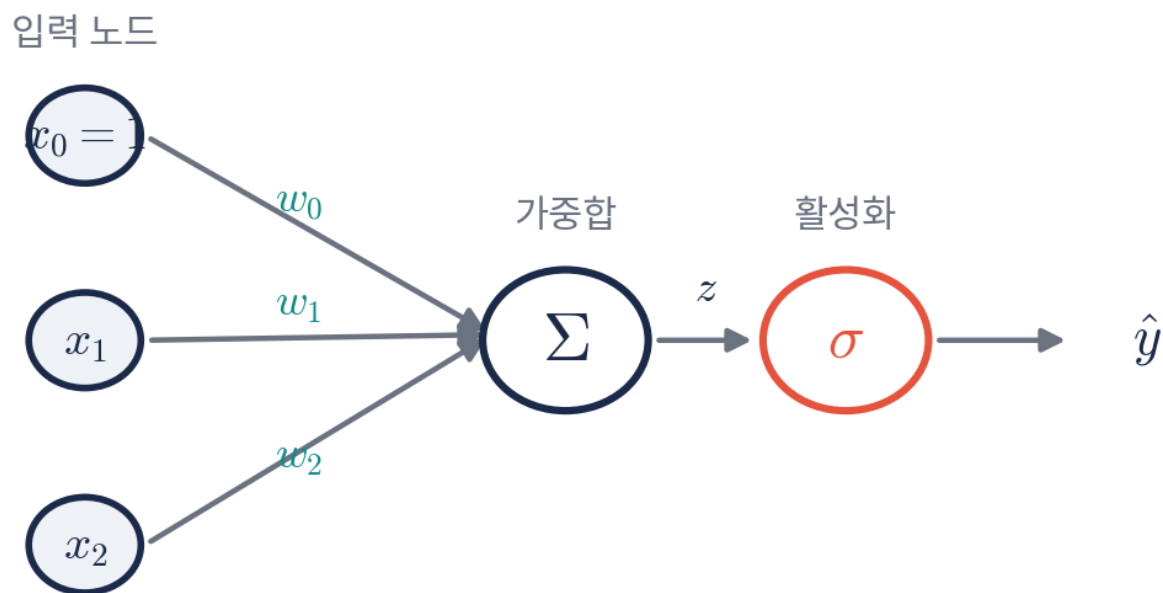
각 픽셀 값 0-16 →  $x_{ij}$

N = 1,797개 이미지 →  $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}$

$y_i \in \{0, \dots, 9\}$  → 실습에서는 "3 vs. 나머지" 이진 문제로 변환

관례: 바이어스를 가중치 벡터에 포함시키기 위해  $x_0 = 1$  성분을 추가하여 (d+1)차원으로 취급

## 퍼셉트론의 구조 (2/3) — 도식적 표현

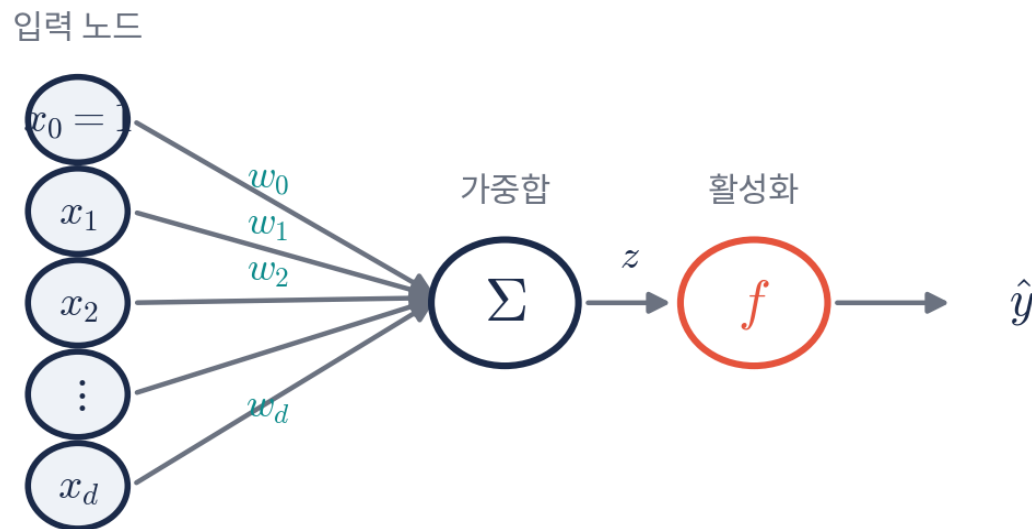


- **노드(node) ○ : 값을 보관하는 단위**
  - 입력 노드: 특징 벡터의 개별 성분( $x_0 = 1$ 은 바이어스 노드)
  - 가중합 노드  $\Sigma$ :  $z = w^T x$
  - 활성화 노드  $f$ :  $\hat{y} = f(z)$
- **에지(edge) → : 정보의 흐름 방향**
  - 각 에지는 가중치(weight)  $w_k$ 를 가지며 "성분  $k$ 의 기여도"를 표현
  - 퍼셉트론 학습 = 가중치  $w$ 의 조정

참고: 퍼셉트론과 다층 신경망 모두에서 모델의 그림이 곧 계산 그래프(computational graph)이다 — PyTorch의 autograd가 따라가는 바로 그 그래프

※ 바이어스 노드( $x_0 = 1$ )가 명시되지 않은 그림에서도 가중합을 구성하는 항의 수 = 특징 벡터 성분 수 + 1

# 퍼셉트론의 구조 (3/3) — 일반형과 가중치의 의미



$$z = \mathbf{w}^T \mathbf{x} = \sum_{k=0}^d w_k x_k = w_0 + w_1 x_1 + \cdots + w_d x_d \quad (x_0 = 1)$$

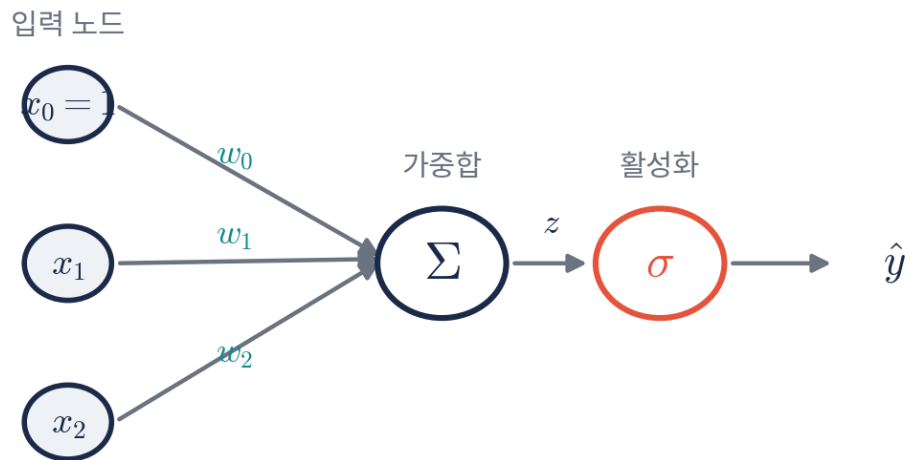
- 입력: d개의 성분 + 바이어스 노드  $x_0 = 1 \rightarrow d+1$ 개 입력 노드
- 가중치  $\mathbf{w} = [w_0 \ w_1 \ \cdots \ w_d]^T$  : 학습 대상 파라미터
  - $w_0$ : 결정 경계의 위치(절편),  $w_1 \dots w_d$ : 경계의 방향(법선 벡터)
  - $|w_k|$ 가 크면 성분 k가 판단에 미치는 영향이 큼
- 파라미터 수 = d+1. 트랜스포머의 FFN 은닉 뉴런 1개도 정확히 d+1개 파라미터

## 이해도 확인

$x_1, x_2, x_3$ 의 세 개 성분으로 개체의 특징 벡터가 구성된다고 하자. 아래 조건을 만족하면 1을, 그렇지 않으면 0을 출력하는 퍼셉트론을 그려 보시오. ( $w_0$ 는 무엇인가? 활성화 함수는?)

$$0.3x_1 + 0.2x_2 - 1.7x_3 > 0.5 \Rightarrow 1, \text{ else } 0$$

# 퍼셉트론의 동작 (1/3) — 가중합과 활성화



$$\mathbf{x} = [1, x_1, x_2]^T, \quad z = w_0 + w_1x_1 + w_2x_2, \quad \hat{y} = \sigma(z)$$

- 입력된 특징 벡터  $\mathbf{x} = [1, x_1, x_2]^T$  ( $x_0 = 1$ 은 바이어스)
- (1) 가중합:  $z = \mathbf{w}^T \mathbf{x} = w_0 + w_1x_1 + w_2x_2$
- (2) 활성화:  $\hat{y} = f(z)$ . 예제에서는 시그모이드  $\rightarrow \hat{y} = \sigma(z) \in (0,1)$
- (3) 결정:  $\hat{y} > 0.5$  이면 클래스 1 ( $= z > 0$ )

## 수치 예제

$$\mathbf{w} = [-1.0, 2.0, 0.5]^T, \quad \mathbf{x} = [1, 0.8, -0.4]^T$$

$$z = -1.0 + 2.0 \cdot 0.8 + 0.5 \cdot (-0.4) = -1.0 + 1.6 - 0.2 = 0.4$$

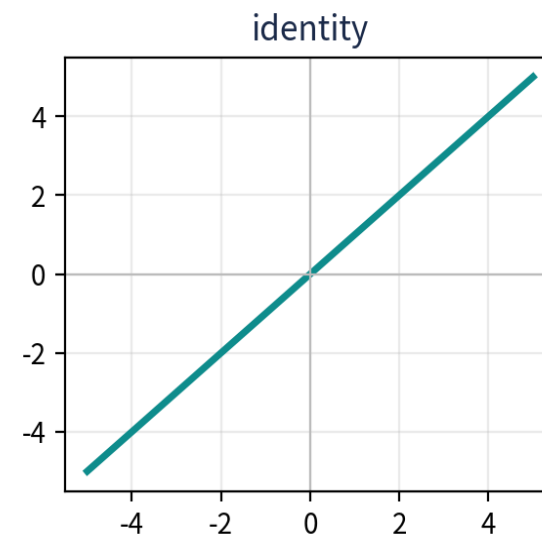
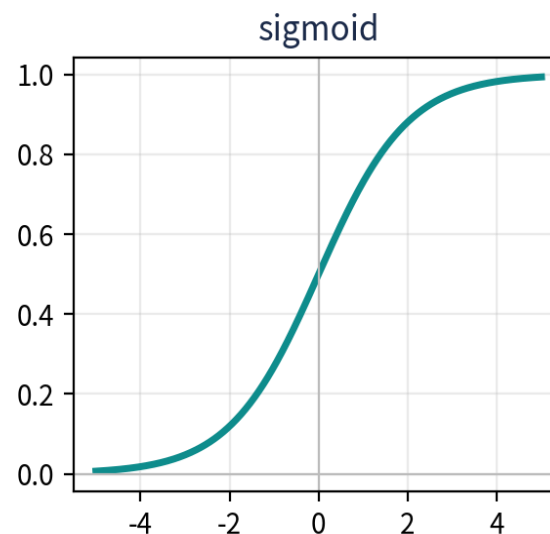
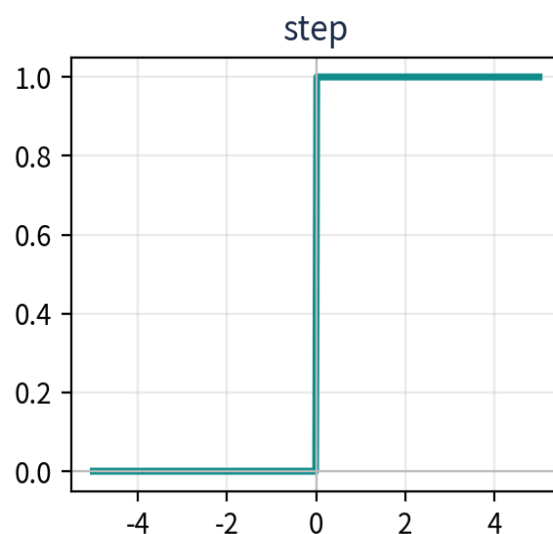
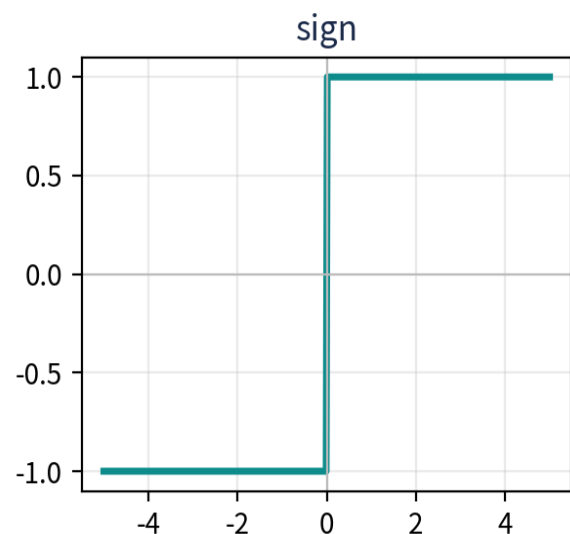
$$\hat{y} = \sigma(0.4) = 1/(1+e^{(-0.4)}) \approx 0.599 \rightarrow \text{클래스 1}$$

같은  $z$ 에 대해  $\text{sgn}(z) = +1$ ,  $\text{step}(z) = 1$ ,  $\text{identity}(z) = 0.4$  : 활성화 함수만 다르다

# 퍼셉트론의 동작 (2/3) — 고전 활성화 함수

$$\text{sgn}(z) = \begin{cases} +1 & z \geq 0 \\ -1 & z < 0 \end{cases}$$

$$\text{step}(z) = \mathbf{1}[z \geq 0] \quad f(z) = z$$



## 부호 함수 sgn

Rosenblatt 원형  
미분 불가 → 오분류 규칙으로 학습

## 계단 함수 step

논리 게이트 표현  
출력 {0,1}, 미분 불가

## 시그모이드 $\sigma$

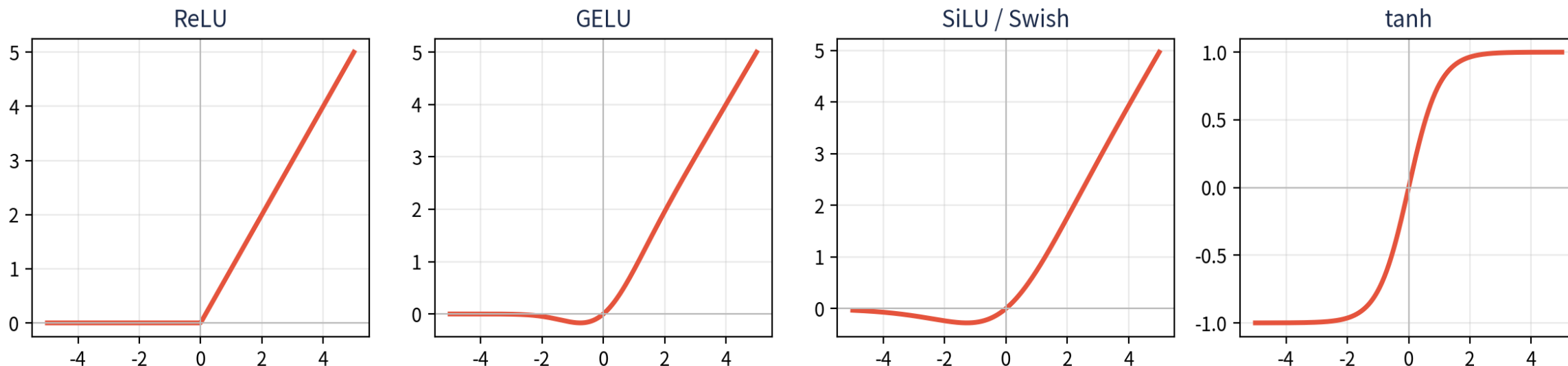
로지스틱 회귀와 동일  
미분 가능, 확률 해석, 기울기 소실

## 항등 함수

선형 회귀와 동일  
MSE 손실과 결합

## 퍼셉트론의 동작 (3/3) — 현대 신경망의 활성화 함수 [최신]

$$\text{ReLU}(z) = \max(0, z) \quad \text{GELU}(z) = z \Phi(z) \quad \text{SiLU}(z) = z \sigma(z) \quad \text{SwiGLU}(\mathbf{x}) = \text{SiLU}(W_1 \mathbf{x}) \odot W_2 \mathbf{x}$$



- **ReLU (2010-)**: 기울기 소실 완화, 희소 활성화. AlexNet·ResNet의 표준
- **GELU (2016)**: BERT·GPT-2/3·ViT의 FFN 활성화. 확률적 게이팅  $z \cdot \Phi(z)$
- **SiLU/Swish, SwiGLU (2020-)**: LLaMA·PaLM·Gemma·DeepSeek 등 최신 LLM FFN의 표준
- 퍼셉트론의 정의는 그대로: 가중합 → 비선형. 바뀐 것은  $f$ 의 선택뿐 (위 그림의 네 번째 패널은 비교용 tanh — SwiGLU는 게이트 구조라 1변수 곡선으로 그릴 수 없다)

# 퍼셉트론의 순전파 (feedforward propagation) 정리

- 순전파: 입력 특징 벡터가 출력 값으로 변환되는 일련의 과정
  - 특징 벡터  $\mathbf{x}$ , 그에 대한 퍼셉트론 출력  $\hat{y}$ , 활성화 함수  $f$  라고 하면 다음과 같이 표현
  - 예제(2개 성분, 시그모이드 활성화)의 출력은 아래와 같음
- 벡터화:  $N$ 개 샘플을 한 번에  $\rightarrow \hat{\mathbf{Y}} = \mathbf{f}(\mathbf{X}\mathbf{w})$ ,  $\mathbf{X} \in \mathbb{R}^{N \times (d+1)}$ 
  - GPU에서 행렬곱 한 번 — PyTorch의 `nn.Linear(d, 1)`가 정확히 이것

$$\hat{y} = f(\mathbf{w}^\top \mathbf{x}) = f\left(\sum_{k=0}^d w_k x_k\right)$$

$$g(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}, \quad P(C_1 | \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}}$$

```
import torch, torch.nn as nn
perceptron = nn.Linear(in_features=2, out_features=1) # w1, w2 + bias(w0)
x = torch.tensor([[0.8, -0.4]])
z = perceptron(x) # 가중합 z = w^T x + b
y_hat = torch.sigmoid(z) # 활성화 (sigmoid → 로지스틱 회귀)
print(z, y_hat)
```

# B

## 퍼셉트론의 학습

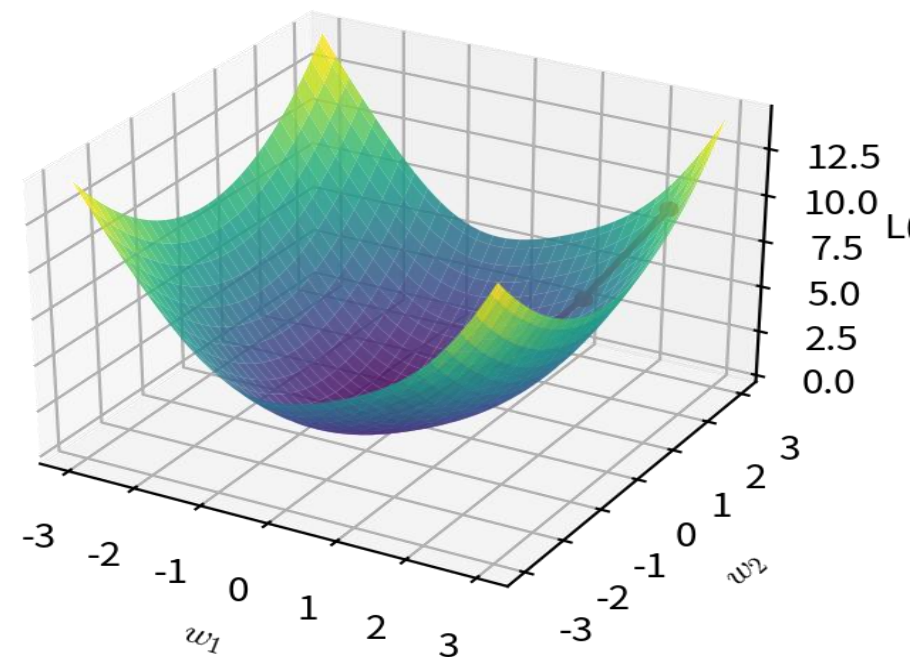
Rosenblatt 학습 규칙 → 경사 하강 → 손실 함수별 편미분 → 수렴 정리와 마진 (대학원 심화)



# 퍼셉트론 학습 (1/6) — 학습 대상과 경사 하강

- 퍼셉트론의 학습 대상: 가중치  $\mathbf{w} = (w_0, w_1, \dots, w_d)$
- 가중치 갱신 방법
  - 1. 초기화: 정해진 구간에 존재하는 임의의 수로 가중치 값을 결정 (PyTorch: Kaiming/Xavier)
  - 2. 개별 가중치  $w_k$ 의 값을 손실  $L(\mathbf{w})$ 가 감소하는 방향으로 반복 갱신
  - $\eta$ : 학습률 ( $0 < \eta < 1$ ). 너무 크면 발산, 너무 작으면 느림
  - $L(\mathbf{w})$ : 최소화하려는 손실 함수 — 활성화 함수와 함께 문제 유형을 결정
  - $\partial L / \partial w_k$ :  $L(\mathbf{w})$ 를  $w_k$ 에 대하여 편미분한 결과 (기울기)

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial L(\mathbf{w})}{\partial \mathbf{w}}, \quad 0 < \alpha < 1$$



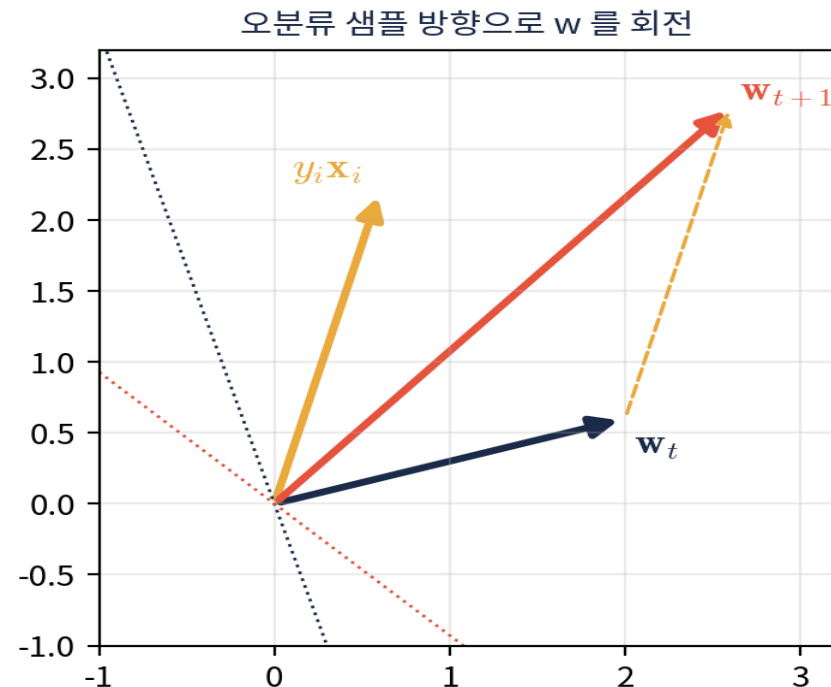
손실 곡면 위에서 기울기의 반대 방향으로 이동 (경사 하강)

핵심 질문: 기울기의 방향으로 조금씩 움직이면 정말 최소점에 도달하는가? → 볼록성(convexity)이 답을 결정

## 퍼셉트론 학습 (2/6) — Rosenblatt 학습 규칙 (1958)

$$\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i \quad \text{if } y_i \mathbf{w}^\top \mathbf{x}_i \leq 0, \quad y_i \in \{-1, +1\}$$

- 출력을  $y \in \{-1, +1\}$ 로 두면 "올바른 분류"  $\Leftrightarrow y_i \cdot \mathbf{w}^\top \mathbf{x}_i > 0$
- 오분류된 샘플에 대해서만  $\mathbf{w}$ 를  $y_i \mathbf{x}_i$  방향으로 이동 ( $\eta$ 는 보통 1)
  - 갱신 후 같은 샘플의 점수:  $y_i(\mathbf{w} + \eta y_i \mathbf{x}_i)^\top \mathbf{x}_i = y_i \mathbf{w}^\top \mathbf{x}_i + \eta \|\mathbf{x}_i\|^2 \rightarrow$  증가
- 이 규칙은 "퍼셉트론 손실"에 대한 SGD와 정확히 같다
  - 손실이 0이 될 때(모두 올바르게 분류)까지 반복 — 종료 조건이 곧 수렴



$$L_{\text{perc}}(\mathbf{w}) = \sum_{i \in \mathcal{M}} -y_i \mathbf{w}^\top \mathbf{x}_i, \quad \nabla_{\mathbf{w}} L_{\text{perc}} = - \sum_{i \in \mathcal{M}} y_i \mathbf{x}_i$$

$\mathbf{w}$ 가 오분류 샘플 쪽으로 회전  $\rightarrow$  결정 경계(점선)가 샘플을 올바른 편으로 넘김

학습률  $\eta$ 는 수렴 여부·갱신 횟수에 영향이 없다 (초기 가중치  $\mathbf{w} = 0$  이면  $\eta$ 는 단순히  $\mathbf{w}$ 의 스케일). 실습 과제 (a)

# 퍼셉트론 학습 (3/6) — 시그모이드 + 교차 엔트로피 = 로지스틱 회귀

- 예제: 활성화 함수  $\sigma$ , 손실 = 이진 교차 엔트로피 (음의 로그 가능도)

– 데이터  $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}$ ,  $\mathbf{x}_i = [1, x_{i1}, x_{i2}]^T$ ,  $y_i \in \{0,1\}$ ,  $\hat{y}_i = \sigma(\mathbf{w}^T \mathbf{x}_i)$

- $\sigma'(z) = \sigma(z)(1-\sigma(z))$  를 이용하면 편미분이 놀랍도록 단순해진다

–  $\partial L / \partial w_k = -(1/N) \sum_i (y_i - \hat{y}_i) x_{ik}$  — "오차 × 입력"

–  $w_0$ :  $x_{i0} = 1$ ,  $w_1$ :  $x_{i1}$ ,  $w_2$ :  $x_{i2}$  를 곱한 항의 평균

$$L(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N \left[ y_i \ln \sigma(\mathbf{w}^T \mathbf{x}_i) + (1 - y_i) \ln (1 - \sigma(\mathbf{w}^T \mathbf{x}_i)) \right]$$

$$\frac{\partial L}{\partial w_k} = -\frac{1}{N} \sum_{i=1}^N (y_i - \sigma(\mathbf{w}^T \mathbf{x}_i)) x_{ik}, \quad w_k \leftarrow w_k + \frac{\alpha}{N} \sum_i (y_i - \hat{y}_i) x_{ik}$$

## 성분별 갱신식 ( $\eta$ : 학습률)

$$w_0 \leftarrow w_0 + (\eta/N) \sum_i (y_i - \hat{y}_i) \cdot 1$$

$$w_1 \leftarrow w_1 + (\eta/N) \sum_i (y_i - \hat{y}_i) \cdot x_{i1}$$

$$w_2 \leftarrow w_2 + (\eta/N) \sum_i (y_i - \hat{y}_i) \cdot x_{i2}$$

주의: 활성화 함수와 손실 함수에 따라 구체적인 편미분 결과가 변한다. 그러나  $\sigma$ +CE, identity+MSE, softmax+CE 모두 " $(\hat{y}-y) \cdot x$ " 꼴이다.

볼록성:  $\sigma$ +CE 손실은  $w$ 에 대해 볼록  $\rightarrow$  경사 하강이 전역 최적에 수렴 (Rosenblatt 규칙과 달리 분리 불가능한 데이터에서도 안정)

## 퍼셉트론 학습 (4/6) — 항등 활성화 + MSE = 선형 회귀

- **예제: 활성화 함수 = 항등 함수, 손실 = 평균 제곱 오차 → 회귀 문제**
  - 데이터  $\mathcal{D} = \{(x_i, y_i)\}$ ,  $y_i \in \mathbb{R}$ ,  $\hat{y}_i = w^T x_i$
- **편미분은 역시 "오차 × 입력" 꼴**
  - $\partial L / \partial w_0 = (2/N) \sum_i (\hat{y}_i - y_i)$ ,  $\partial L / \partial w_1 = (2/N) \sum_i (\hat{y}_i - y_i) x_{i1}$ ,  $\partial L / \partial w_2 = (2/N) \sum_i (\hat{y}_i - y_i) x_{i2}$
- **MSE는  $w$ 에 대한 2차 함수 → 닫힌 해  $w^* = (X^T X)^{-1} X^T y$  도 존재 (정규 방정식)**
  - 하지만  $d$ 가 크면  $O(d^3)$  → 경사 하강/SGD가 실용적. 이것이 딥러닝이 GD를 쓰는 이유

$$L(w) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2, \quad \frac{\partial L}{\partial w_k} = \frac{2}{N} \sum_{i=1}^N (\hat{y}_i - y_i) x_{ik}$$

### 세 가지 퍼셉트론, 하나의 갱신식

sgn + 퍼셉트론 손실 → 분류 (Rosenblatt)  
 $\sigma$  + 교차 엔트로피 → 확률적 분류 (로지스틱)  
identity + MSE → 회귀 (선형 회귀)

$$\text{공통: } \Delta w \propto \sum_i (y_i - \hat{y}_i) x_i$$

일반화 선형 모형(GLM) 관점: 활성화  $f$  = 연결 함수의 역함수, 손실 = 대응 지수족의 NLL 일 때 기울기는 항상  $(y - \hat{y})x$

# 퍼셉트론 학습 (5/6) — 배치·온라인·미니배치, 그리고 현대 옵티마이저 [최신]

## 배치 모드 학습

학습 데이터를 구성하는 모든 샘플에 대하여 목표 값과 출력 값의 차이를 계산한 후 가중치를 한 번 갱신. 정확한 기울기, 느린 1회 갱신, 메모리 부담.

## 온라인 모드 학습

매 특징 벡터에 대하여 목표 값과 출력 값의 차이를 계산한 후 가중치를 갱신 (SGD). 잡음이 크지만 빠르고, 국소 최소점 탈출에 유리. Rosenblatt 규칙이 이 방식.

## 미니배치 (현대 표준)

B개(32-4096)씩 묶어 갱신. GPU 병렬성과 기울기 분산의 절충. LLM 사전학습은 수백만 톤 단위 배치 + 기울기 누적.

옵티마이저의 진화: SGD → Momentum → Adam(2015) → AdamW(2019) → Muon(2024-25)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad \mathbf{w}_t = \mathbf{w}_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \alpha \lambda \mathbf{w}_{t-1}$$

AdamW: 1.2차 모멘트로 성분별 학습률 조정 + 분리된 가중치 감쇠( $\lambda$ ). GPT·LLaMA 계열의 기본 옵티마이저

## Muon (2024-25)

행렬 파라미터의 기울기를 직교화(Newton-Schulz)하여 갱신. 2025년 Kimi K2 등 대규모 LLM 사전학습에 채택. 이론: 스펙트럴 노름 최대 마진 (Part B, 암묵적 편향)

```
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-2) # 미니배치 루프 안에서: opt.zero_grad(); loss.backward(); opt.step()
```

## 퍼셉트론 학습 (6/6) — 수렴 정리 (Novikoff 1962) [대학원 심화]

정리. 학습 데이터가 마진  $\gamma$ 로 선형 분리 가능하고 모든 특징 벡터의 노름이  $R$  이하이면, Rosenblatt 규칙( $\eta = 1$ , 초기 가중치  $\mathbf{w} = 0$ )의 갱신 횟수는  $(R/\gamma)^2$  을 넘지 않는다.

$$\exists \mathbf{w}^*, \|\mathbf{w}^*\| = 1, \gamma > 0 : \quad y_i \mathbf{w}^{*\top} \mathbf{x}_i \geq \gamma \quad \forall i, \quad \|\mathbf{x}_i\| \leq R \quad \Rightarrow \quad \# \text{updates} \leq \left( \frac{R}{\gamma} \right)^2$$

### 증명 (세 줄)

①  $\mathbf{w}^*$ 와의 내적은 갱신마다  $\gamma$  이상 증가 (오분류 샘플  $\mathbf{x}_i$ 에 대해  $y_i \mathbf{x}_i^\top \mathbf{w}^* \geq \gamma$ ):

$$\mathbf{w}_{t+1}^\top \mathbf{w}^* = \mathbf{w}_t^\top \mathbf{w}^* + y_i \mathbf{x}_i^\top \mathbf{w}^* \geq \mathbf{w}_t^\top \mathbf{w}^* + \gamma \quad \Rightarrow \quad \mathbf{w}_T^\top \mathbf{w}^* \geq T\gamma$$

② 노름의 제곱은 갱신마다  $R^2$  이하로만 증가 (오분류이므로  $y_i \mathbf{w}_t^\top \mathbf{x}_i \leq 0$ ):

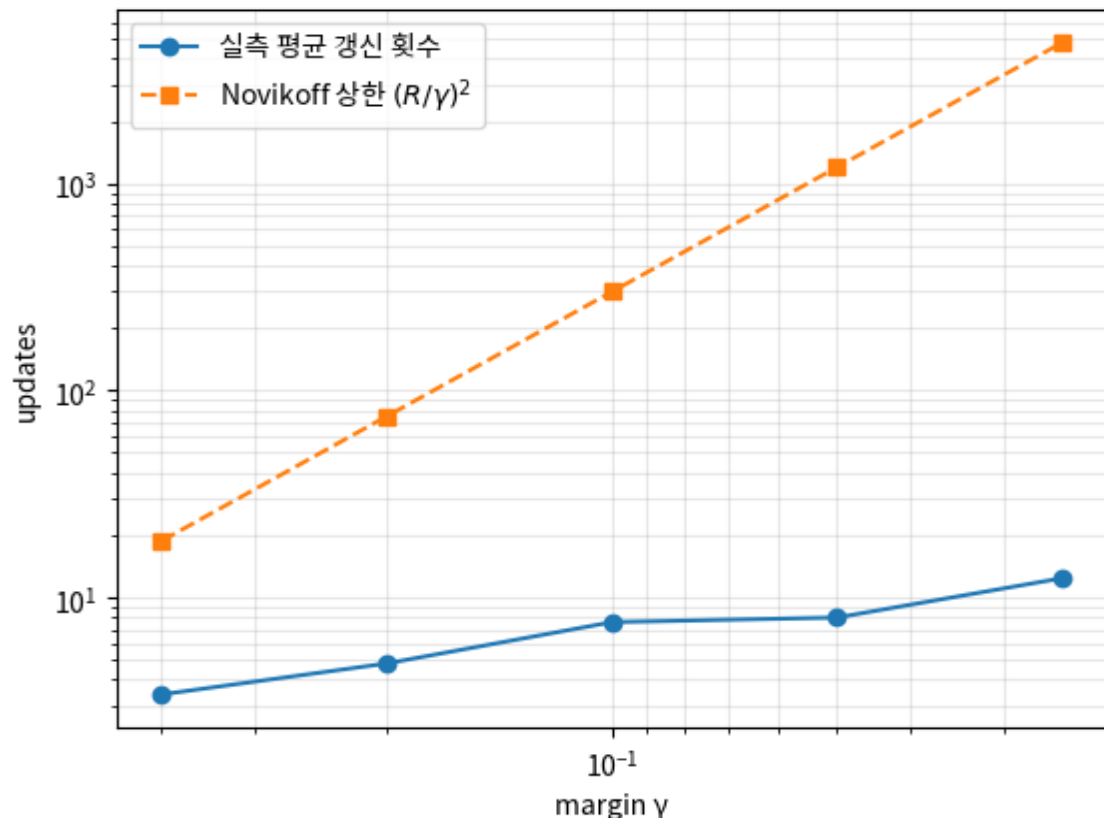
$$\|\mathbf{w}_{t+1}\|^2 = \|\mathbf{w}_t\|^2 + 2y_i \mathbf{w}_t^\top \mathbf{x}_i + \|\mathbf{x}_i\|^2 \leq \|\mathbf{w}_t\|^2 + R^2 \quad \Rightarrow \quad \|\mathbf{w}_T\|^2 \leq TR^2$$

③ 코시-슈바르츠 부등식으로 두 결과를 결합:

$$T\gamma \leq \mathbf{w}_T^\top \mathbf{w}^* \leq \|\mathbf{w}_T\| \|\mathbf{w}^*\| \leq \sqrt{T} R \quad \Rightarrow \quad T \leq \frac{R^2}{\gamma^2}$$

■ 상한은 데이터 개수  $N$ , 차원  $d$ 와 무관하며 오직  $\text{기하}(R, \gamma)$ 에만 의존한다.  $\eta \neq 1$ 이어도 동일 ( $\mathbf{w}$ 가  $\eta$ 배 스케일).

# 마진 $\gamma$ 와 갱신 횟수 — 실험으로 확인 [대학원 심화]



log-log (x축은 오른쪽으로 갈수록  $\gamma$ 가 작아지도록 뒤집혀 있음): 마진이 절반  $\rightarrow$  상한 4배. 실측(파랑)은 항상 상한(주황) 아래

| margin $\gamma$ | 평균 갱신 횟수 | $(R/\gamma)^2$ 상한 |
|-----------------|----------|-------------------|
| 0.400           | 3.4      | 18.7              |
| 0.200           | 4.8      | 75.0              |
| 0.100           | 7.6      | 300.0             |
| 0.050           | 8.0      | 1200.0            |
| 0.025           | 12.4     | 4800.0            |

- 실습 노트북 파트 1: 2차원,  $N = 200$ , seed 10회 평균,  $R = \sqrt{3}$
- 상한은 최악의 샘플 순서에 대한 보장 — 실제로는 훨씬 적은 갱신으로 수렴
- $\gamma \rightarrow 0$  (거의 겹치는 데이터)이면 갱신 횟수가 폭발 — 분리 불가능하면 영원히 진동
- 해법: 마진을 "최대화"하는 경계를 찾자  $\rightarrow$  SVM, 그리고 놀랍게도 로지스틱 회귀의 GD

## 마진·SVM·암묵적 편향(implicit bias) — 2018 → 2025 [최신 이론]

$$\gamma(\mathbf{w}) = \min_i \frac{y_i \mathbf{w}^\top \mathbf{x}_i}{\|\mathbf{w}\|}, \quad \text{SVM : } \max_{\mathbf{w}} \gamma(\mathbf{w}) \iff \min \|\mathbf{w}\|^2 \text{ s.t. } y_i \mathbf{w}^\top \mathbf{x}_i \geq 1$$

- 퍼셉트론은 분리 초평면을 "아무거나" 찾는다 (초기값·순서 의존). 어느 경계가 좋은가?
  - 최대 마진 경계(hard-margin SVM)가 일반화 오차 상한이 가장 작다 — 마진 기반 일반화 이론
- **Soudry, Hoffer, Nacson, Gunasekar, Srebro (JMLR 2018):** 분리 가능 데이터에서 로지스틱 손실의 GD는 명시적 정규화 없이도 최대 마진 방향으로 수렴
  - 손실은 0으로 가지만  $w$ 의 노름은  $\ln t$ 로 발산 — 방향만 수렴. 수렴 속도  $O(1/\ln t)$ 로 매우 느림

$$\lim_{t \rightarrow \infty} \frac{\mathbf{w}(t)}{\|\mathbf{w}(t)\|} = \frac{\hat{\mathbf{w}}_{\text{SVM}}}{\|\hat{\mathbf{w}}_{\text{SVM}}\|}, \quad \mathbf{w}(t) = \hat{\mathbf{w}}_{\text{SVM}} \ln t + O(1)$$

### 왜 중요한가

과매개화된 신경망이 훈련 오차 0인 수많은 해 중 "왜 잘 일반화하는 해"를 고르는지 설명하는 첫 단서. 옵티마이저가 곧 정규화기다.

### 최근 확장 (2024–2025)

다중 클래스 (NeurIPS 2024): softmax·교차 엔트로피 GD도 다중 클래스 SVM 방향으로 수렴

Fan, Schmidt, Thrampoulidis (2025): Muon·스펙트럴 디센트는 스펙트럴 노름 기준 최대 마진, Adam류(max-norm 디센트)는  $\infty$ -노름 기준 최대 마진 — 옵티마이저마다 다른 해로 간다

# 표현력과 일반화 — VC 차원, 그리고 세 선형 분류기 비교 [대학원 심화]

$$\text{VC}(\text{perceptron}, \mathbb{R}^d) = d + 1,$$

$$R(h) \leq \hat{R}(h) + \sqrt{\frac{(d + 1) \left( \ln \frac{2N}{d+1} + 1 \right) + \ln \frac{4}{\delta}}{N}}$$

d차원 퍼셉트론은 일반 위치의 d+1개 점을 임의로 나눌 수 있지만 d+2개는 불가 (Radon 정리). 일반화 오차 상한은  $\sqrt{((d+1)/N)}$  로 감소 — 파라미터 수 대비 데이터가 충분해야 한다는 고전적 결론.

|           | 퍼셉트론 (Rosenblatt)         | 로지스틱 회귀                                              | 선형 SVM                                         |
|-----------|---------------------------|------------------------------------------------------|------------------------------------------------|
| 손실        | $\max(0, -y \cdot w^T x)$ | $\ln(1 + e^{-(y \cdot w^T x)})$                      | $\max(0, 1 - y \cdot w^T x) + \lambda \ w\ ^2$ |
| 볼록성       | 볼록 (비강볼록)                 | 볼록, 매끄러움                                             | 강볼록                                            |
| 출력        | 클래스 ( $\pm 1$ )           | 확률 $P(y x)$                                          | 클래스 · 마진 점수                                    |
| 분리 가능 시 해 | 임의의 분리 초평면 (유한 단계)        | GD $\rightarrow$ 최대 마진 방향 ( $t \rightarrow \infty$ ) | 최대 마진 (명시적)                                    |
| 분리 불가능 시  | 진동, 미수렴                   | 수렴 (유일해)                                             | 수렴 (소프트 마진)                                    |
| 현대적 후손    | FFN 뉴런, 힌지형 손실            | softmax 출력층, 교차 엔트로피                                 | 마진 기반 이론, 대조 손실                                |

**결론:** 세 모델의 차이는 손실 함수 하나. 딥러닝의 출력층은 거의 항상 "로지스틱/softmax 퍼셉트론"이다.

C

# 논리 게이트와 퍼셉트론

AND · OR의 퍼셉트론 표현 → 결정 경계의 기하 → XOR과 단일 퍼셉트론의 한계



# 논리 게이트의 표현 (1/6) — 진리표

- 논리 게이트의 동작을 퍼셉트론으로 설명하는 예제는 매우 유명한 예제
- AND, OR, XOR 게이트와 같은 논리 게이트는 2개의 성분으로 구성된 특징 벡터를 입력으로 받아 하나의 값으로 구성된 출력을 생성
- 논리 게이트의 입력과 출력 값은 기본적으로 불린 값(Boolean value)이나 계산의 편의를 위해 "True"는 1로, "False"는 0으로 표현
- 따라서 학습 데이터는 단 4개의 샘플 ( $N = 4, d = 2$ )

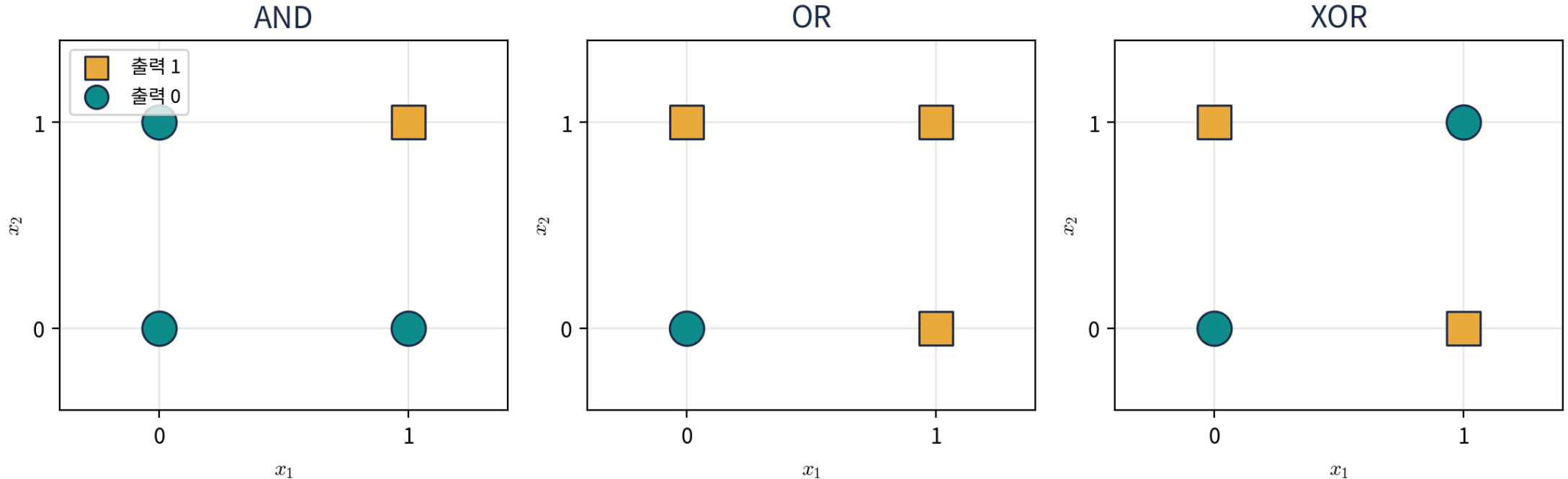
| 구분      | $x_1$ | $x_2$ | AND | OR | XOR |
|---------|-------|-------|-----|----|-----|
| $i = 1$ | 0     | 0     | 0   | 0  | 0   |
| $i = 2$ | 0     | 1     | 0   | 1  | 1   |
| $i = 3$ | 1     | 0     | 0   | 1  | 1   |
| $i = 4$ | 1     | 1     | 1   | 1  | 0   |

XOR: 두 입력이 서로 다를 때만 1 — 이 열 하나가 신경망 역사를 바꿨다

목표:  $w_0, w_1, w_2$  세 숫자만으로 각 열을 재현하는 퍼셉트론  $z = w_0 + w_1x_1 + w_2x_2, y = \text{step}(z)$  를 찾을 수 있는가?

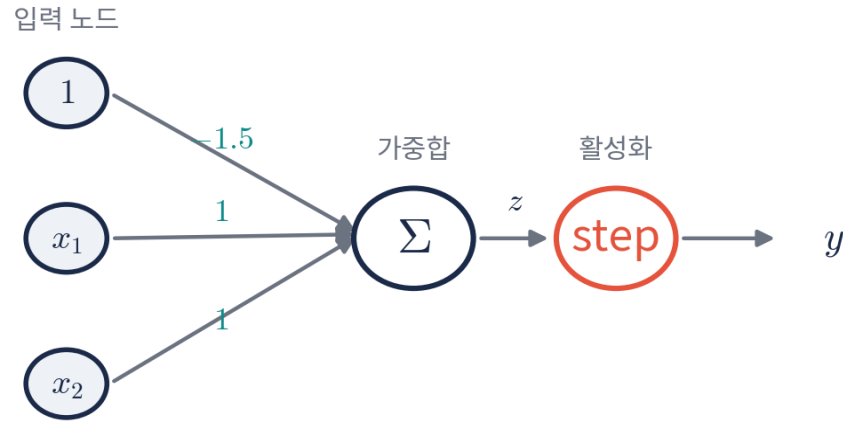
# 논리 게이트의 표현 (2/6) — 시각적 표현

AND, OR, XOR 게이트의 시각적 표현: 입력과 출력 모두 1 혹은 0의 값으로 구성



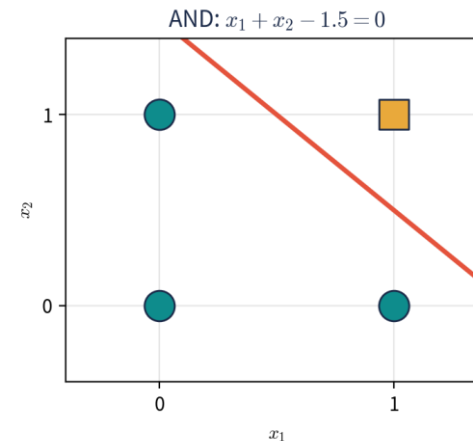
- 출력 1, ● 출력 0. 특징 공간  $(x_1, x_2)$  위의 4개 점을 두 집단으로 나누는 직선이 존재하는가?
- AND: (1,1)만 분리 → 가능. OR: (0,0)만 분리 → 가능. XOR: 대각선 두 점씩 → 직선 하나로는 불가능

# 논리 게이트의 표현 (3/6) — AND 게이트



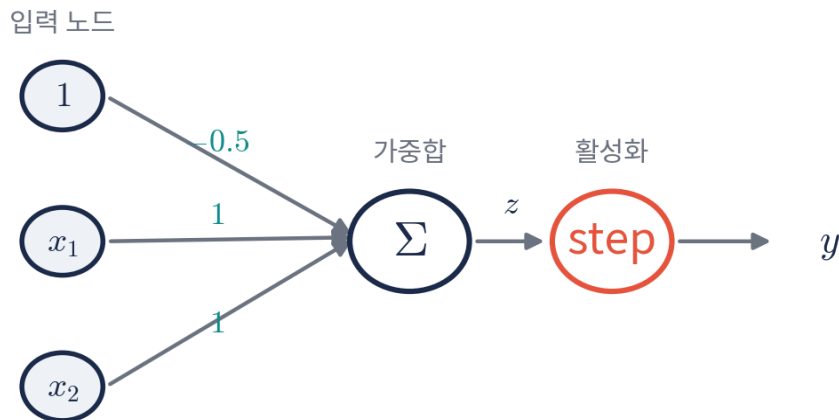
- 퍼셉트론을 활용한 논리 게이트의 표현: 특징 성분의 가중합  $z = \mathbf{w}^T \mathbf{x}$  로 표현할 때
  - $x_1 = 0, x_2 = 0$  이면  $z = -1.5 < 0 \rightarrow$  출력 0
  - $x_1 = 0, x_2 = 1$  이면  $z = -0.5 < 0 \rightarrow$  출력 0
  - $x_1 = 1, x_2 = 0$  이면  $z = -0.5 < 0 \rightarrow$  출력 0
  - $x_1 = 1, x_2 = 1$  이면  $z = 0.5 > 0 \rightarrow$  출력 1
- 계단 함수를 적용하면 ( $z > 0 \Rightarrow 1$ ) 정확히 AND. 시그모이드를 써도  $\sigma(z) > 0.5 \Leftrightarrow z > 0$  이므로 동일
- 해는 유일하지 않다:  $w_0 \in (-2, -1)$ ,  $w_1 = w_2 = 1$  이면 모두 AND — 실습에서 학습된  $\mathbf{w} = [-4, 3, 2]$ 도 AND

$$z = x_1 + x_2 - 1.5 \quad (w_0 = -1.5, w_1 = w_2 = 1)$$



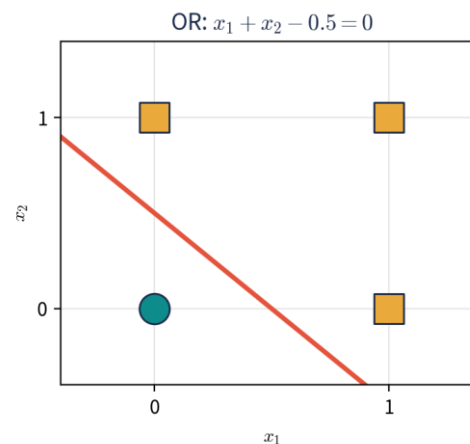
빨간 직선:  $x_1 + x_2 - 1.5 = 0$  (결정 경계). 경계 위쪽( $z > 0$ )에 (1,1)만 존재

# 논리 게이트의 표현 (4/6) — OR 게이트



- OR 게이트의 표현: 절편만  $-1.5 \rightarrow -0.5$  로 바꾸면 된다
  - $x_1 = 0, x_2 = 0$  이면  $z = -0.5 < 0 \rightarrow$  출력 0
  - $x_1 = 0, x_2 = 1$  이면  $z = 0.5 > 0 \rightarrow$  출력 1
  - $x_1 = 1, x_2 = 0$  이면  $z = 0.5 > 0 \rightarrow$  출력 1
  - $x_1 = 1, x_2 = 1$  이면  $z = 1.5 > 0 \rightarrow$  출력 1
- 같은 방향( $w_1 = w_2 = 1$ )의 경계를 평행 이동한 것 —  $w_0$ 가 "문턱값 (threshold)" 역할
- NAND: 모든 부호를 뒤집어  $z = -x_1 - x_2 + 1.5$ . NOT:  $z = -x + 0.5$ . 이 셋으로 모든 논리 회로가 만들어진다

$$z = x_1 + x_2 - 0.5 \quad (w_0 = -0.5, w_1 = w_2 = 1)$$

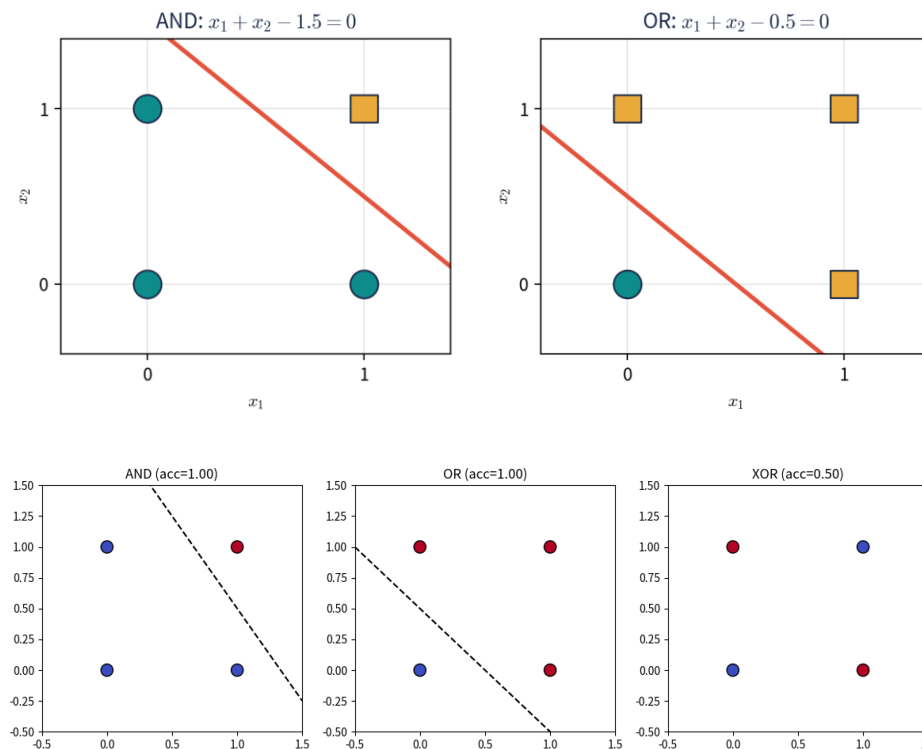


빨간 직선:  $x_1 + x_2 - 0.5 = 0$ . 경계 아래쪽( $z < 0$ )에 (0,0)만 존재

# 논리 게이트의 표현 (5/6) — 결정 경계의 기하학

$$z = \mathbf{w}^T \mathbf{x} = 0 \iff x_2 = -\frac{w_1}{w_2}x_1 - \frac{w_0}{w_2}$$

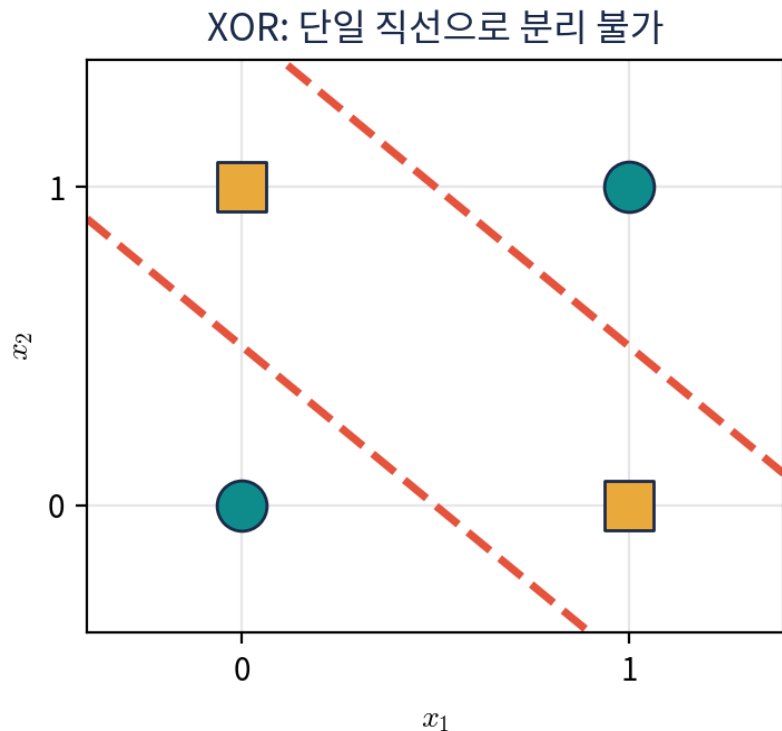
- $z = \mathbf{w}^T \mathbf{x} = \sum_k w_k x_k$  : 특징 벡터 성분의 가중합
- $z = 0$  : (1) 특징 성분의 가중합으로 만들어지는 직선 (2) 논리 게이트의 "1" 출력과 "0" 출력을 구분할 수 있게 특징 공간을 분할
  - 법선 벡터  $(w_1, w_2)$ 가 경계의 방향을,  $|w_0|/||(w_1, w_2)||$ 가 원점으로부터의 거리를 결정
  - d차원에서는 초평면(hyperplane) — 퍼셉트론 = "초평면 하나로 공간을 두 쪽으로 자르는 기계"
- 점  $x$ 에서 경계까지의 부호 있는 거리 =  $\mathbf{w}^T \mathbf{x} / ||(\mathbf{w}_1, \dots, \mathbf{w}_d)||$  — 이것이 곧 마진 (Part B)



위: 손으로 설계한 경계. 아래: Rosenblatt 규칙으로 학습된 경계(실습 파트 2) — XOR은 100 에폭 후에도  $w = 0$  으로 되돌아옴

$z = 0$  : 논리 게이트의 "1" 출력과 "0" 출력을 구분하는 구분선. 이 직선 하나가 존재하지 않는 문제가 XOR

## 논리 게이트의 표현 (6/6) — XOR과 단일 퍼셉트론의 한계



- XOR 게이트에서 "0"의 출력과 "1"의 출력을 구분하려면 2개의 가중합으로 만들어진 직선이 필요
  - AND 게이트와 OR 게이트의 경우처럼 단일 퍼셉트론을 이용하여 XOR 게이트의 동작을 표현할 수 없음
- 증명 (귀류법):  $\text{step}(w_0 + w_1x_1 + w_2x_2)$ 가 XOR라면  $w_0 < 0$ ,  $w_0 + w_2 > 0$ ,  $w_0 + w_1 > 0$ ,  $w_0 + w_1 + w_2 < 0$ . 가운데 둘을 더하면  $2w_0 + w_1 + w_2 > 0$ , 첫·셋째를 더하면  $2w_0 + w_1 + w_2 < 0$  — 모순
- Minsky & Papert, Perceptrons (1969): 이 한계를 엄밀히 분석 → "1차 AI 겨울"의 한 원인
- 퍼셉트론의 동작을 개선할 필요가 있음 → 여러 개의 퍼셉트론을 결합한 새로운 모델 (다층 신경망)

중요한 구분: 이것은 학습 알고리즘의 실패가 아니라 가설 공간(선형 분류기)의 표현력 한계다. 데이터가 아무리 많아도, 에폭이 아무리 길어도 해결되지 않는다.

※ 커널 트릭(SVM)이나 특징 추가( $x_1x_2$  항)로도 해결 가능 — 신경망은 그 특징 변환 자체를 "학습"한다는 점이 다르다

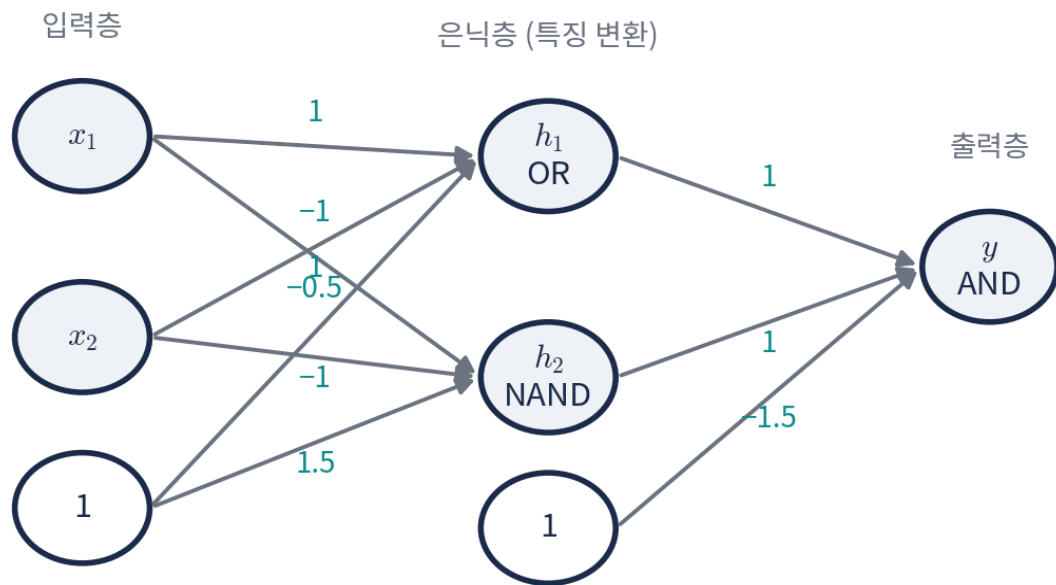
# D

## 다층 퍼셉트론 (MLP)

은닉층으로 XOR 해결 → 특징 학습 → 보편 근사 정리 → 역전파 → PyTorch 실습



# 은닉층 하나로 XOR 풀기 — 퍼셉트론 3개의 결합

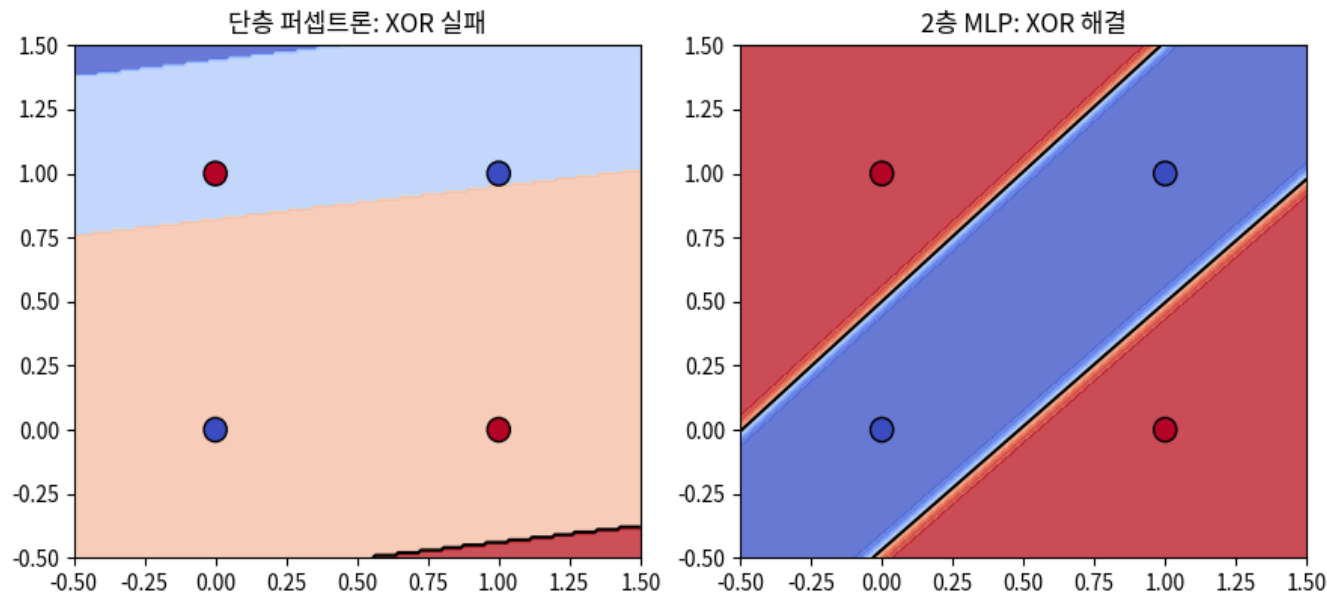


- $XOR = (x_1 \text{ OR } x_2) \text{ AND NOT}(x_1 \text{ AND } x_2) = OR \wedge NAND$ 
  - 은닉 뉴런  $h_1 = \text{OR}$  퍼셉트론,  $h_2 = \text{NAND}$  퍼셉트론 (각각 Part C에서 설계한 것)
  - 출력 뉴런 =  $h_1, h_2$ 에 대한 AND 퍼셉트론
- 계산:  $(0,0) \rightarrow h=(0,1) \rightarrow 0$ ,  $(0,1) \rightarrow (1,1) \rightarrow 1$ ,  $(1,0) \rightarrow (1,1) \rightarrow 1$ ,  $(1,1) \rightarrow (1,0) \rightarrow 0$  ✓
- 파라미터 9개( $3 \times 3$ )로 XOR 완성. 실습 파트 4: tanh 은닉 2개로 손실 0.0001까지 학습됨

$$h_1 = \text{step}(x_1 + x_2 - 0.5) \text{ [OR]}, \quad h_2 = \text{step}(-x_1 - x_2 + 1.5) \text{ [NAND]}, \quad y = \text{step}(h_1 + h_2 - 1.5) \text{ [AND]}$$

핵심: 은닉층이 입력을 새로운 좌표계 ( $h_1, h_2$ )로 옮기면 XOR은 그 공간에서 선형 분리 가능해진다. 출력층은 여전히 "퍼셉트론"이다.

# 은닉층 = 학습된 특징 변환 (실습 파트 4 결과)



$$\mathbf{h} = \phi(W_1\mathbf{x} + \mathbf{b}_1), \quad \hat{y} = f(\mathbf{w}_2^\top \mathbf{h} + b_2)$$

- 왼쪽: 단층 퍼셉트론 — 2000 에폭 후에도 손실 0.693 (=  $\ln 2$ , 무작위 추측 수준)
- 오른쪽: 2→2→1 MLP — 두 개의 은닉 뉴런이 만든 두 직선의 "사이"가 클래스 0
- 학습된 은닉 출력  $\mathbf{h}(\mathbf{x})$ :  $(0,0) \rightarrow (-.99, -.98)$ ,  $(0,1) \rightarrow (.99, -.1)$ ,  $(1,0) \rightarrow (-1, .99)$ ,  $(1,1) \rightarrow (-.99, -.99)$ 
  - $\mathbf{h}$ -공간에서는 클래스 0 두 점이 한쪽 모서리에 몰려 직선 하나로 분리됨

## 이 구조가 딥러닝 전체의 골격

CNN: 합성곱 층이 특징 변환 → 마지막 FC 층이 softmax 퍼셉트론

트랜스포머: 어텐션+FFN 층이 특징 변환 → 출력층(unembedding)이 softmax 퍼셉트론

**"표현 학습(representation learning)" = 마지막 퍼셉트론이 풀기 쉬운 공간을 찾는 일**

**실습 4-b: digits 데이터에서 선형(softmax) 0.960 → 2층 MLP(GELU) 0.971 — 은닉층 하나의 값**

# 보편 근사 정리와 깊이의 이점 [대학원 심화]

정리 (Cybenko 1989, Hornik 1991). 은닉층 하나와 비다항 활성화 함수  $\phi$ 를 갖는 MLP는 컴팩트 집합 위의 임의의 연속 함수를 원하는 정밀도로 근사할 수 있다.

$$\forall g \in C([0, 1]^d), \varepsilon > 0 \quad \exists m, W_1, \mathbf{b}_1, \mathbf{w}_2 : \sup_{\mathbf{x}} \left| g(\mathbf{x}) - \mathbf{w}_2^\top \phi(W_1 \mathbf{x} + \mathbf{b}_1) \right| < \varepsilon$$

## 존재 ≠ 학습 가능

정리는 은닉 폭  $m$ 의 크기를 말하지 않는다 — 최악의 경우  $m$ 은  $\varepsilon^{-d}$ 로 폭발(차원의 저주). 또한 GD가 그 해를 찾는다는 보장도 없다.

## 깊이의 이점 (2014–)

ReLU 망의 선형 영역 수는 깊이에 지수적으로 증가(Montúfar 2014). 얇은 망이 지수적 폭을 요구하는 함수를 깊은 망은 다항 크기로 표현 (Telgarsky 2016).

## 2024–26 관점

트랜스포머는 "깊이 × 폭 × 데이터"의 스케일링 법칙을 따르며, 한 층 FFN의 폭(4d)과 층 수(L)의 비율이 경험적으로 조정된다. 보편 근사는 출발점일 뿐.

퍼셉트론 → MLP로 "표현력"은 해결. 남은 문제는 "어떻게 학습하는가" → 역전파

## 역전파(backpropagation) 한눈에 — 연쇄 법칙의 효율적 구현

$$\delta^{(L)} = \nabla_{\hat{y}} L \odot f'(z^{(L)}), \quad \delta^{(l)} = (W_{l+1}^\top \delta^{(l+1)}) \odot \phi'(z^{(l)}), \quad \frac{\partial L}{\partial W_l} = \delta^{(l)} \mathbf{h}^{(l-1)\top}$$

- 순전파: 층마다  $z^{(l)} = W_l \mathbf{h}^{(l-1)} + b_l$ ,  $\mathbf{h}^{(l)} = \phi(z^{(l)})$  를 계산하며 중간값을 저장
- 역전파: 출력층의 오차 신호  $\delta^{(L)}$ 에서 시작하여  $W^\top$ 를 곱하고  $\phi'$ 를 곱하며 입력 쪽으로 전파
- 각 층의 기울기 = "오차 신호  $\times$  그 층의 입력" — 퍼셉트론의  $(y - \hat{y})x$  규칙이 층마다 반복되는 것
- 비용: 순전파와 같은  $O(\# \text{파라미터})$ . 수치 미분(파라미터당 순전파 1회)보다 수백만 배 효율적
- Rumelhart, Hinton & Williams (1986) — Minsky-Papert 이후 17년 만에 다층망 학습을 실용화

```
# PyTorch: autograd가 역전파를 자동 수행
mlp = nn.Sequential(nn.Linear(2, 2), nn.Tanh(),
                    nn.Linear(2, 1))
opt = torch.optim.AdamW(mlp.parameters(), lr=0.1)
for epoch in range(2000):
    logits = mlp(X) # 순전파
    loss = F.binary_cross_entropy_with_logits(logits, y)
    opt.zero_grad() # 역전파 (δ 계산)
    loss.backward()
    opt.step() # w ← w - α · ∂L/∂w
```

기울기 소실/폭발:  $\phi'$ 를 L번 곱하므로  $|\phi'| < 1$  (sigmoid  $\leq 0.25$ )이면 앞쪽 층 기울기가 사라진다 → ReLU, 잔차 연결, LayerNorm이 해결책

# 실습 — sklearn · NumPy · PyTorch로 퍼셉트론 구현

```
# [파트 1] Rosenblatt 학습 규칙 (NumPy)
def perceptron_train(X, y, lr=1.0, max_epochs=1000):
    w = np.zeros(X.shape[1]); n_updates = 0
    for epoch in range(max_epochs):
        errors = 0
        for xi, yi in zip(X, y):
            if yi * (w @ xi) <= 0:      # 오분류일 때만
                w += lr * yi * xi
                n_updates += 1; errors += 1
        if errors == 0: break
    return w, n_updates

# [파트 3] scikit-learn 1.8 (기존 실습 갱신)
clf = make_pipeline(StandardScaler(),
                    Perceptron(tol=1e-3, random_state=0))
clf.fit(X_train, y_train)      # digits, one-vs-rest 자동
ConfusionMatrixDisplay.from_predictions(y_test, clf.predict(X_test))
```

```
# [파트 4] PyTorch 2.x - 퍼셉트론과 MLP
import torch, torch.nn as nn, torch.nn.functional as F
single = nn.Linear(2, 1)      # 단층 퍼셉트론
mlp = nn.Sequential(nn.Linear(2, 2), nn.Tanh(),
                    nn.Linear(2, 1))      # 2층 MLP

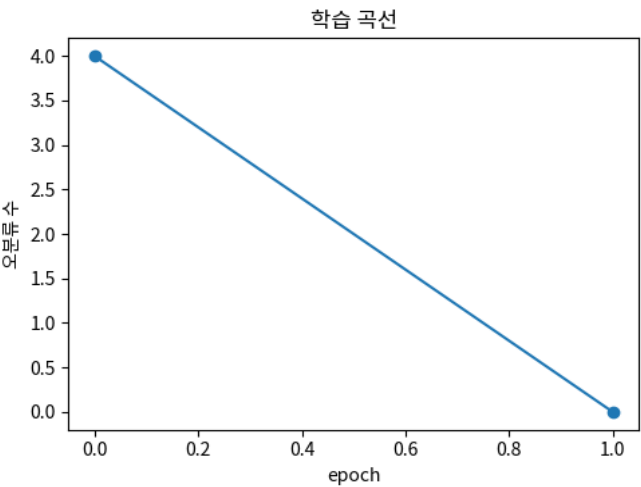
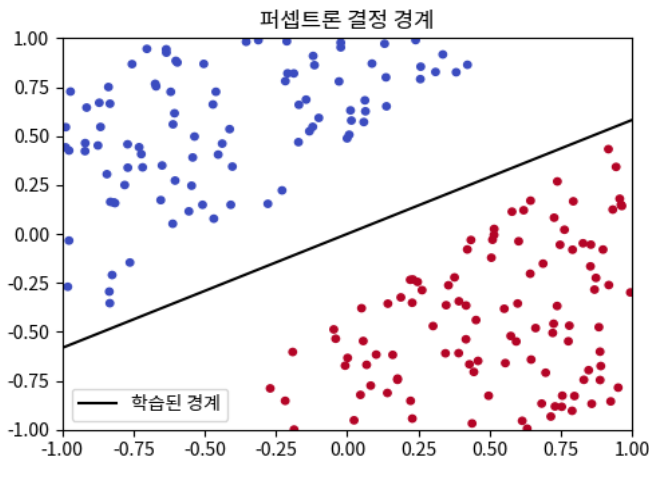
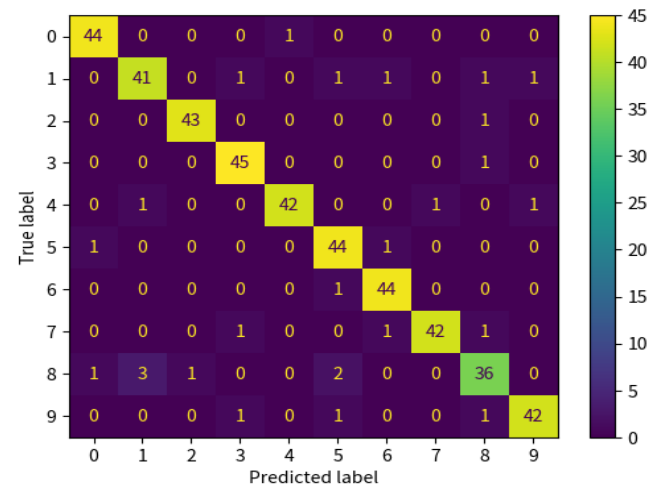
def train(model, X, y, epochs=2000, lr=0.1):
    opt = torch.optim.AdamW(model.parameters(), lr=lr)
    for ep in range(epochs):
        loss = F.binary_cross_entropy_with_logits(model(X), y)
        opt.zero_grad(); loss.backward(); opt.step()
    return model

train(single, X_xor, y_xor)    # loss 0.693, acc 0.50~0.75
train(mlp, X_xor, y_xor)      # loss 0.0001, acc 1.00

# [파트 5] 트랜스포머 안의 MLP
layer = nn.TransformerEncoderLayer(64, nhead=4, dim_feedforward=256)
# linear1 (64→256) + linear2 (256→64) = FFN, 파라미터의 66%
```

실행 환경: Python ≥ 3.10, numpy · scikit-learn 1.8 · matplotlib · torch 2.x (CPU로 충분, 전체 실행 1분 내외). 노트북에는 실행 결과와 그림이 포함되어 있습니다.

# 실습 결과 요약



sklearn Perceptron, digits 테스트 정확도 0.940 (표준화 파이프 라인)

Rosenblatt 규칙: 마진 0.3 데이터에서 4회 갱신, 2 에폭 만에 수렴

| 모델                           | 문제             | 결과                          | 해석             |
|------------------------------|----------------|-----------------------------|----------------|
| NumPy 퍼셉트론                   | AND / OR / XOR | 1.00 / 1.00 / 0.50          | XOR은 표현력 한계    |
| sklearn Perceptron (OvR)     | digits 10클래스   | acc 0.940                   | 선형 분류기의 기준선    |
| PyTorch nn.Linear(2,1)       | XOR            | loss 0.693, acc $\leq 0.75$ | 볼록 손실이어도 해 없음  |
| PyTorch MLP 2→2→1 (tanh)     | XOR            | loss 1e-4, acc 1.00         | 은닉 2개로 충분      |
| PyTorch softmax 퍼셉트론 64→10   | digits         | acc 0.960                   | AdamW + 표준화    |
| PyTorch MLP 64→128→10 (GELU) | digits         | acc 0.971                   | 은닉층 1개로 +1.1%p |

E

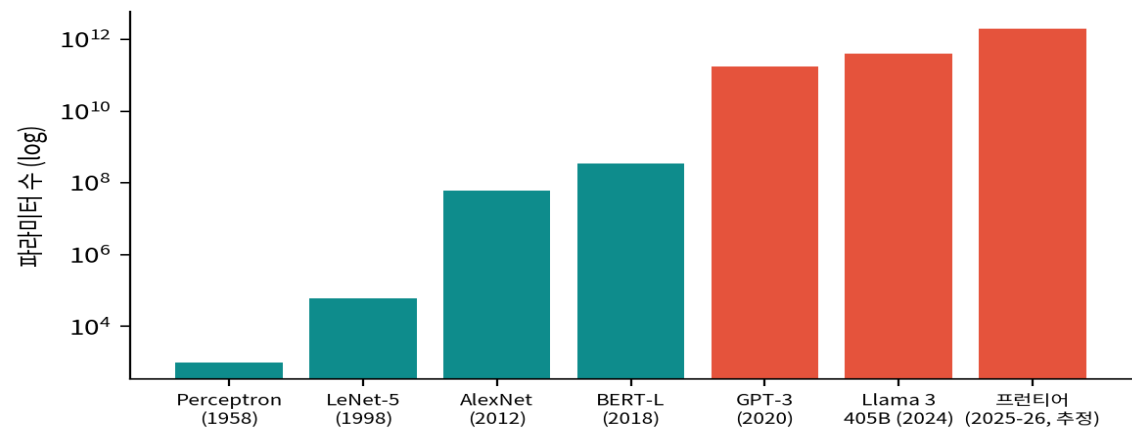
# 퍼셉트론에서 트랜스포머·LLM으로

어텐션 = 학습된 가중합 → FFN = 2층 MLP → 다음 토큰 예측 → 사전학습·정렬·RAG·에이전트  
(LLM 자료 통합)



# 계보: 1958년 수식이 2026년 모델 안에 있다

| 모델              | 핵심 수식                                      | 퍼셉트론과의 관계         | 학습             |
|-----------------|--------------------------------------------|-------------------|----------------|
| 퍼셉트론 (1958)     | $z = w^T x, \hat{y} = f(z)$                | 뉴런 1개, 파라미터 $d+1$ | Rosenblatt 규칙  |
| MLP (1986)      | $h = \varphi(W_1 x), \hat{y} = f(w_2^T h)$ | 퍼셉트론을 층으로 쌓음      | 역전파 + SGD      |
| CNN (1998–2012) | 가중치 공유 퍼셉트론(필터)                            | 국소 수용장 + 풀링       | 역전파 + GPU      |
| 트랜스포머 (2017)    | 어텐션 + FFN(=MLP) $\times L$                 | 학습된 가중합으로 토큰 혼합   | AdamW + 스케일링   |
| LLM (2020–26)   | $p(x_t   x_{<t}) = \text{softmax}(W h_t)$  | 수천억 파라미터, 수조 토큰   | 사전학습 + 정렬 + RL |



- 파라미터 수:  $10^3$  (퍼셉트론)  $\rightarrow 10^{12}$  (2026 프런티어, 추정) —  $10^9$ 배
- 그러나 각 파라미터가 하는 일은 여전히 "곱하고 더하고 비선형 통과"
- 달라진 것: 뉴런의 연결 방식(어텐션), 규모, 데이터, 최적화·정렬 기법

# 어텐션 = 학습된 가중합 — 퍼셉트론의 일반화

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \quad \mathbf{o}_i = \sum_j \alpha_{ij} \mathbf{v}_j, \quad \alpha_{ij} = \frac{e^{\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d_k}}}{\sum_{j'} e^{\mathbf{q}_i^\top \mathbf{k}_{j'} / \sqrt{d_k}}}$$

## 퍼셉트론의 가중합

$z = \sum_k w_k x_k$  — 가중치  $w_k$ 는 학습 후 고정. 어떤 입력이 와도 같은  $w$ 를 사용.

## 어텐션의 가중합

$\mathbf{o}_i = \sum_j \alpha_{ij} \mathbf{v}_j$  — 가중치  $\alpha_{ij}$ 가 입력(쿼리·키)에 따라 매번 계산됨. "입력 의존적 가중치"를 가진 퍼셉트론.

- $Q = W_Q x, K = W_K x, V = W_V x$  — 세 개의 선형(퍼셉트론) 사영. 학습되는 것은  $W_Q, W_K, W_V$
- $\text{softmax}(\cdot)$  는 "어느 토큰을 볼지"를 결정하는 미분 가능한 선택기 — 비선형 활성화의 역할
- 멀티헤드: 서로 다른  $(W_Q, W_K, W_V)$  세트를  $h$ 개 병렬 실행 → 여러 관점의 가중합
- 2024-26 변형: GQA/MQA (키·값 공유), MLA (DeepSeek, 저차원 압축), 슬라이딩 윈도우·선형 어텐션 — 모두  $\alpha_{ij}$  계산 비용을 줄이려는 시도

# 트랜스포머 블록 안의 퍼셉트론 — FFN은 2층 MLP



$$\text{FFN}(\mathbf{x}) = W_2 \phi(W_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2, \quad W_1 \in \mathbb{R}^{4d \times d}, W_2 \in \mathbb{R}^{d \times 4d}$$

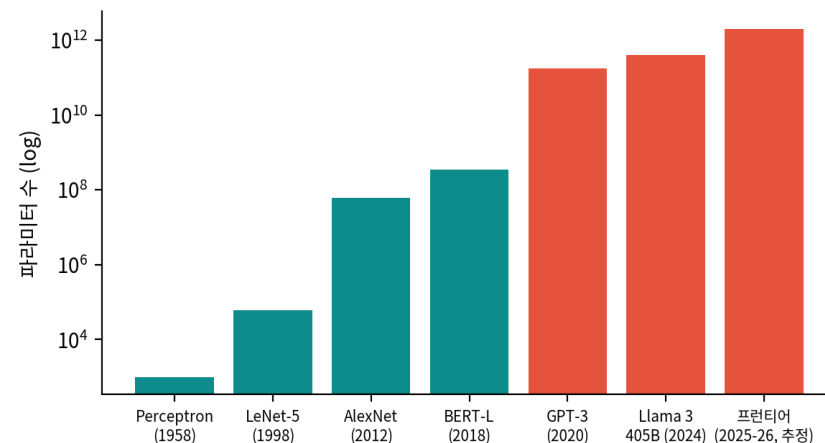
- 각 토큰 벡터  $\mathbf{x} \in \mathbb{R}^d$ 가 독립적으로 FFN을 통과 — 폭 4d의 은닉층을 가진 MLP
  - 은닉 뉴런 1개 = 퍼셉트론 1개:  $w_1^T \mathbf{x} + b \rightarrow \phi \rightarrow$  출력 가중치. GPT-3( $d=12288$ )는 층당 49,152개 퍼셉트론
- 파라미터 비율: FFN  $8d^2$  vs 어텐션  $4d^2 \rightarrow$  전체의 약 2/3가 "퍼셉트론 더미"
  - 실습 파트 5: nn.TransformerEncoderLayer(64, 4, 256)에서 linear1+linear2 = 66.2%
- 해석 연구(2021-): FFN 뉴런은 키-값 메모리처럼 동작 — 특정 패턴(키)에 발화해 특정 토큰 분포(값)를 더함
- MoE (Mixture of Experts, 2024-26 표준): FFN을 여러 "전문가 MLP"로 나누고 토큰마다 일부만 활성화 — DeepSeek-V3/V4, Llama 4, Qwen3, Gemini 등

SwiGLU FFN(LLaMA 계열):  $\phi$  대신  $\text{SiLU}(W_1 \mathbf{x}) \odot W_3 \mathbf{x}$  — 게이트가 붙은 퍼셉트론. 활성화 함수만 바뀌었다.

# 규모의 법칙 — 퍼셉트론 $10^3$ 개에서 $10^{12}$ 개로

$$L(N, D) \approx E + \frac{A}{N^\alpha} + \frac{B}{D^\beta} \quad (\text{Chinchilla : } D_{\text{opt}} \approx 20 N)$$

- Kaplan et al. (2020), Hoffmann et al. (Chinchilla, 2022): 손실이 파라미터  $N$ ·데이터  $D$ 의 거듭제곱 법칙으로 감소
  - 계산 예산이 주어지면  $N$ 과  $D$ 를 함께 키워야 최적 — 토큰 수  $\approx 20 \times$  파라미터 수
- 2024-26: "추론 시간 스케일링" — 학습 후에도 더 오래 생각(사고 사슬, 탐색)하면 성능 향상 (o1, DeepSeek-R1 계열)
- 퍼셉트론과의 연결: VC 차원 기반 일반화 오차 상한( $\sqrt{((d+1)/N)}$ )은 파라미터가 데이터보다 많으면 무의미하다고 말하지만, 실제 LLM은 그 영역에서 잘 일반화한다 → 암묵적 편향 (Part B) 연구의 동기



## 2026년 9월 현재 지형 (예시)

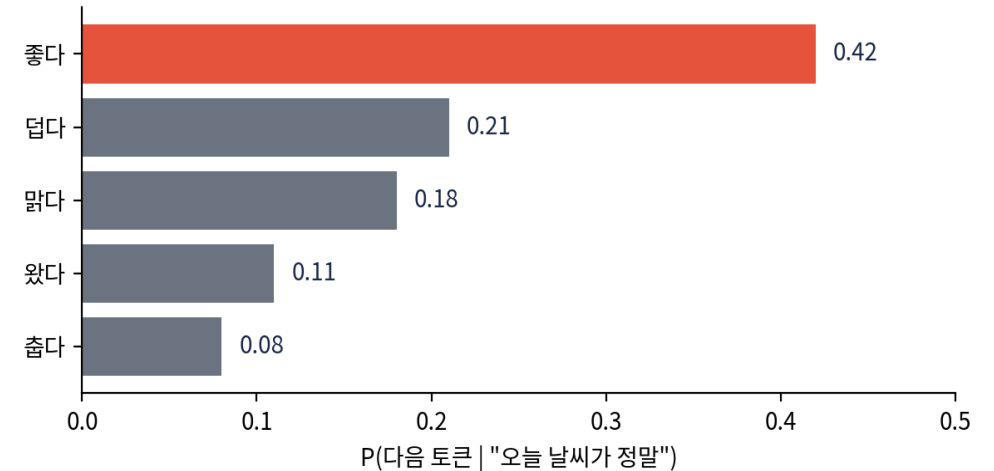
프런티어(비공개): GPT-6 (OpenAI), Gemini 3.x (Google), Claude 5.x (Anthropic)

오픈 웨이트: DeepSeek-V4, Llama, Qwen, Gemma, Mistral 등

2026년 들어 390건 이상의 모델 출시 (llm-stats 집계) — 강의 시점에 최신 표를 직접 확인할 것

# LLM이란? — 다음 토큰 예측이라는 "거대한 퍼셉트론"

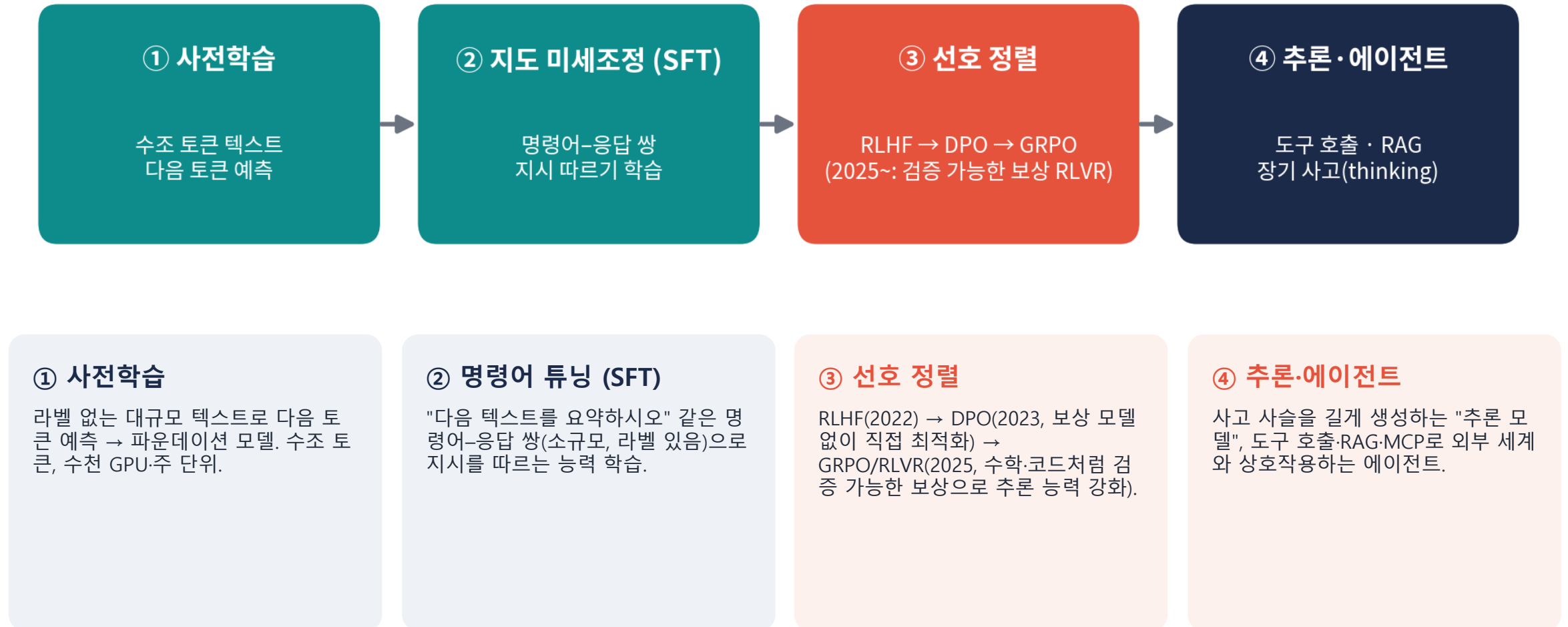
- LLM (Large Language Model): 대규모 텍스트 데이터를 학습해 다양한 언어 작업을 수행하는 인공지능 모델
- 입력 "오늘 날씨가 정말" → 문맥을 분석해 다음 토큰의 확률 분포를 출력 → 가장 확률 높은 "좋다" 선택
- 출력층은 어휘 크기  $V$ (수만~수십만)만큼의 softmax 퍼셉트론:  $\mathbf{h}_t \in \mathbb{R}^d$  에  $W_{\text{out}} \in \mathbb{R}^{V \times d}$ 를 곱함
- 정답이 "좋다"였다면 → 교차 엔트로피 손실 → 역전파로 모든 층의 파라미터가 수정된다
- "My favorite breakfast is \_\_\_" : toast 0.30, bagel 0.20, hamburger 0.10 ...  
— 훈련은 이 확률을 정답 쪽으로 미는 것



$$\max_{\theta} \sum_t \log p_{\theta}(x_t | x_{<t}), \quad p_{\theta}(x_t | x_{<t}) = \text{softmax}(W_{\text{out}} \mathbf{h}_t)$$

학습 과정 = 퍼셉트론 학습과 동일한 원리: 입력·정답 쌍 → 출력 → 손실 → 기울기 → 갱신. 다른 것은 규모와 "정답이 곧 다음 단어"라는 자기지도 (self-supervised) 설정

# LLM의 학습 과정 (2026 기준) — 사전학습 → 미세조정 → 정렬 → 추론



# 사전학습과 미세조정 — 파운데이션 모델

- **사전학습 (pre-training)**
  - 라벨이 붙지 않은 대규모 데이터 → 파운데이션 모델 (foundation model)
  - 텍스트 패턴·세계 지식·추론의 "원재료"를 학습. 비용의 대부분이 여기에 소요
- **미세조정 (fine-tuning)**
  - 라벨이 붙은 소규모 데이터로 특정 과제·도메인에 맞게 조정 → 미세조정된 LLM
  - 전체 파라미터 미세조정 vs. PEFT (LoRA, QLoRA): 저차원 행렬만 학습해 비용 1/100 (11주차 전이학습에서 상세)
- **퍼셉트론 관점**
  - 사전학습 = 특징 변환(은닉층) 학습, 미세조정 = 마지막 퍼셉트론(과 일부 층)의 재학습. 전이 학습의 본질

## LLM vs. ChatGPT — 무엇이 다른가

LLM(파운데이션): 텍스트 패턴을 인식·완성. "한국의 수도는?" → 답이 아니라 계속 이어지는 문장을 생성할 수도 있음

챗봇(ChatGPT류): 사전학습 + 명령어 튜닝 + 정렬 → 명령어에 응답하는 능력을 학습

명령어 튜닝 vs 미세조정: 명령어 튜닝은 "지시 따르기"라는 범용 능력을 위한 미세조정의 한 종류. 많은 (명령어, 응답) 샘플이 필요

# 정렬(alignment): RLHF (2022) → DPO (2023) → GRPO · RLVR (2025) [최신]

1. 초기 모델 생성 (SFT)

2. 초기 답변 데이터 생성

3. 인간 피드백 제공 (선호 순위)

4. 보상 모델 학습

5. 강화 학습 적용 (PPO)

RLHF (Reinforcement Learning from Human Feedback) — InstructGPT/ChatGPT의 방식

$$L_{\text{DPO}} = -\mathbb{E} \left[ \log \sigma \left( \beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

- **DPO (Rafailov et al. 2023):** 보상 모델·RL 없이 선호 쌍 ( $y_w > y_l$ )으로 직접 로지스틱 손실 최적화
  - 보상의 차이를 시그모이드에 넣은 형태 — 정확히 "선호에 대한 로지스틱 회귀(퍼셉트론)"
- **GRPO (DeepSeek, 2024–25):** 같은 질문에 여러 답을 샘플링해 그룹 내 상대 보상으로 정책 갱신 — 가치 모델 불필요
  - RLVR: 수학·코드처럼 정답 검증이 가능한 보상 → 긴 사고 사슬(추론)이 자발적으로 출현 (DeepSeek-R1, 2025)

## 명령어 튜닝 vs RLHF vs RL 추론

명령어 튜닝: "이렇게 답하라"는 예시를 모방 (지도학습)

RLHF/DPO: "어느 답이 더 좋은가"라는 상대 선호를 학습 (인간 가치 정렬)

RLVR/GRPO: "답이 맞았는가"라는 검증 보상으로 문제 해결 과정 자체를 학습 (추론 능력)

2026 프런티어 모델은 세 단계를 모두 거치며, 3단계 비중이 급증하는 추세

# 환각 현상과 RAG (Retrieval-Augmented Generation)

## 환각(hallucination)의 예

"알렉산더 대왕은 언제 미국으로 여행을 떠났나요?" → "알렉산더 대왕은 기원전 324년에 미국으로 탐험을 떠났으며 원주민들과 교류를 나누었습니다."  
— 유창하지만 거짓. 원인: 모델은 확률적으로 그럴듯한 다음 토큰을 생성할 뿐 사실 여부를 검증하지 않는다.

## 환각의 유형과 대응

유형: 사실 왜곡 · 허구의 창조 · 잘못된 추론. 대응: 인간 피드백(RLHF), 지식 접지(Knowledge Grounding), 모델 업데이트, 출처 검증, 그리고 RAG.



- RAG: 질문 → 검색기가 지식 베이스에서 관련 문서 조회 → "질문 + 근거 문서"를 LLM에 입력 → 명확한 출처 기반 답변
- 2025-26 흐름: 긴 컨텍스트(수백만 토큰)와 에이전틱 검색(모델이 스스로 검색 질의를 반복)이 고전적 RAG 파이프라인을 흡수

# LLM 기반 AI 에이전트 — 지각 · 추론 · 행동

## 지각 (Perception)

텍스트·이미지·음성·문서·화면 등 환경 입력을 받아 상태를 파악 (멀티모달 LLM).

## 추론 및 의사결정

목표를 하위 과제로 분해, 계획 수립, 도구 선택. 2025–26 추론 모델의 긴 사고 사슬이 핵심.

## 행동 (Acting)

도구 호출(API·코드 실행·검색·브라우저), 결과 관찰 → 다시 추론 (ReAct 루프).

- 도구 호출 시스템: 입력 → LLM → 도구 모음(도구 1...n) → 출력. 에이전트 시스템: 지시 → 에이전트 → 환경에 행동 → 수행 결과·피드백 → 재추론
- MCP (Model Context Protocol, 2024–): 도구·데이터 소스를 표준 인터페이스로 연결 — 에이전트 생태계의 "USB"
- 멀티 에이전트·컴퓨터 사용(Computer Use)·코딩 에이전트가 2026년 실무 적용의 중심
- 구성 요소 정리 (arXiv 2309.07864): 두뇌(LLM) · 지각 · 행동 · 기억 · 계획

## 퍼셉트론으로 돌아가서

에이전트의 "도구를 호출할지 말지"의 결정도 결국 다음 토큰(예: 함수 이름) 위의 softmax 퍼셉트론이다. 1958년의 "가중합 → 문턱값 → 결정"이라는 구조가 2026년 에이전트의 매 단계에 들어 있다.

F

# 최신 연구 동향 (2024–2026)

KAN · 스파이킹 뉴런과 뉴로모픽 · 옵티마이저의 암묵적 편향 — 퍼셉트론 관점에서 읽기

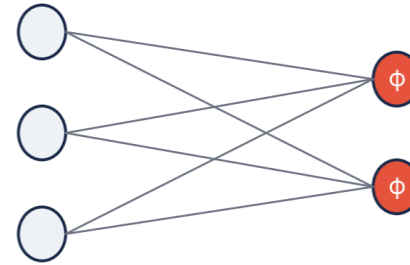


# KAN (Kolmogorov–Arnold Networks, 2024) — "에지에서 학습하는" 뉴런

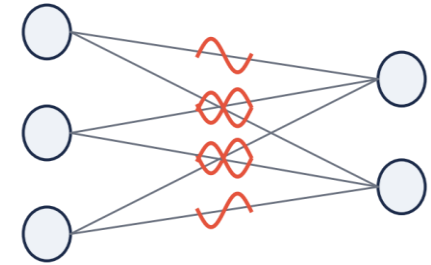
$$f(x_1, \dots, x_d) = \sum_{q=0}^{2d} \Phi_q \left( \sum_{p=1}^d \varphi_{q,p}(x_p) \right)$$

Kolmogorov–Arnold 표현 정리: 다변수 연속 함수는 1변수 함수들의 합성과 덧셈만으로 표현된다

MLP: 노드에 고정 활성화  $\phi$ , 에지에 학습 가중치  $w$



KAN: 에지에 학습 가능한 함수  $\varphi_{ij}(x)$  (B-spline)



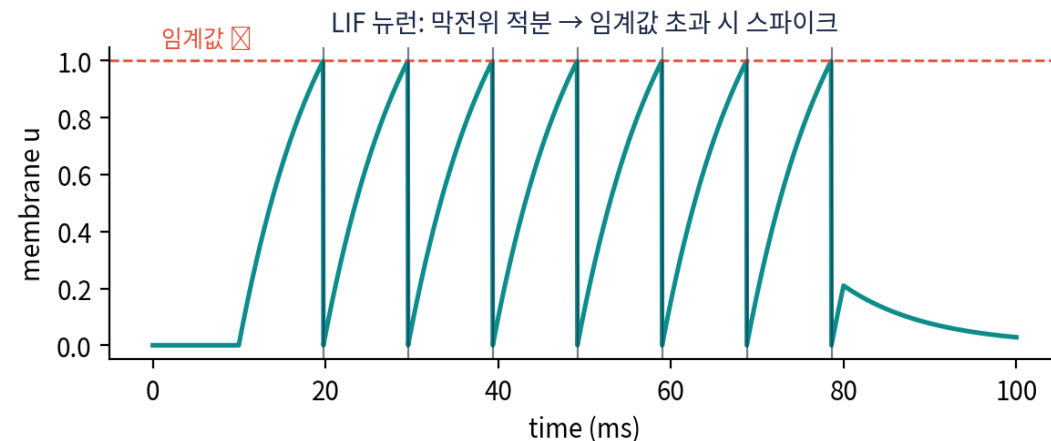
$$\text{MLP : } \mathbf{h} = \phi(W\mathbf{x}) \quad \text{KAN : } h_j = \sum_i \varphi_{ji}(x_i) \quad (\varphi_{ji} : \text{B-spline})$$

- Liu et al. (2024): MLP의 "고정 활성화 + 학습 가중치" 대신 에지마다 학습 가능한 1변수 함수(B-spline)를 둬
- 주장: 더 적은 파라미터로 높은 정확도, 학습된 함수를 기호식으로 읽을 수 있어 해석 가능(과학 발견용)
- 2025년 종합 서베이(arXiv 2407.11075 v8, 2025-09)의 냉정한 평가: KAN이 MLP를 이기는 영역은 기호 회귀뿐, 일반 ML·CV·NLP 벤치마크에서는 지속적으로 열세. 계산 오버헤드 1.36–100배. "차원의 저주 극복" 주장은 엄밀한 근거 부족
- 수업의 교훈: 뉴런의 정의(가중합+비선형)를 바꾸는 시도는 계속되지만, 하드웨어(행렬곱)와 최적화 친화성이 퍼셉트론형 MLP를 살아남게 한다

# 스파이킹 뉴런과 뉴로모픽 컴퓨팅 — 생물학적 퍼셉트론 [최신]

$$\tau \frac{du}{dt} = -(u - u_{\text{rest}}) + RI(t), \quad u \geq \vartheta \Rightarrow \text{spike}, u \leftarrow u_{\text{rest}}$$

- **LIF (Leaky Integrate-and-Fire) 뉴런**: 입력 전류를 시간에 걸쳐 적분하고 임계값  $\vartheta$ 를 넘으면 스파이크를 내며 초기화
  - 퍼셉트론과 비교: 가중합(적분) → 문턱값(step) 구조는 같으나 시간 축이 있고 출력이 이산 이벤트(0/1 시퀀스)
- **장점**: 이벤트 기반 희소 연산으로 에너지 효율 — 뉴로모픽 칩(Intel Loihi 2, IBM NorthPole 등)에서 수십~수백 배 저전력
- **난점**: 스파이크는 미분 불가 → 대리 기울기(surrogate gradient)로 역전파, 또는 ANN→SNN 변환. 정확도는 여전히 ANN에 열세
- **2025-26 동향**: 엣지 AI용 SNN·ANN 비교 리뷰(Frontiers 2025), SNN 아키텍처 탐색 서베이(2025), 스파이킹 트랜스포머 — Rosenblatt이 모사하려 한 뉴런으로 다시 회귀



## 퍼셉트론 세대 구분

- 1세대: McCulloch-Pitts / Rosenblatt — 이진 출력, 문턱값
- 2세대: 시그모이드·ReLU 뉴런 — 연속 출력, 역전파 (현재 딥러닝·LLM)
- 3세대: 스파이킹 뉴런 — 시간 부호화, 이벤트 기반 (뉴로모픽)

세 세대 모두 "가중합 → 비선형 결정"이라는 퍼셉트론의 뼈대를 공유한다

# 퍼셉트론 관점의 최신 이론 — 읽어 볼 논문

| 연도      | 논문                                                                      | 퍼셉트론과의 연결         | 한 줄 결론                                                             |
|---------|-------------------------------------------------------------------------|-------------------|--------------------------------------------------------------------|
| 1962    | Novikoff, On convergence proofs on perceptrons                          | 수렴 정리 원전          | 갱신 횟수 $\leq (R/\gamma)^2$                                          |
| 2018    | Soudry et al., The implicit bias of GD on separable data (JMLR)         | 로지스틱 퍼셉트론의 GD     | 정규화 없이도 최대 마진 방향으로 수렴                                              |
| 2024    | Ravi et al., Implicit bias of GD on separable multiclass data (NeurIPS) | softmax 퍼셉트론      | 다중 클래스에서도 최대 마진 (PERM 손실 일반화)                                      |
| 2024–25 | Liu et al., KAN; Somvanshi et al., KAN 종합 서베이 (v8, 2025)                | 뉴런 정의의 대안         | 기호 회귀 외에는 MLP 우세, 1.36–100× 느림                                     |
| 2025    | Fan, Schmidt, Thrampoulidis, Implicit bias of spectral descent and Muon | 옵티마이저별 마진 기하      | Muon $\rightarrow$ 스펙트럴 노름, Adam류 $\rightarrow$ $\infty$ -노름 최대 마진 |
| 2025    | Frontiers Neurosci., Deep vs spiking NN for edge AI; SNN-NAS survey     | 3세대 뉴런            | 저전력 이벤트 기반 뉴런의 실용화 탐색                                              |
| 2025–26 | 추론 모델(o-시리즈, R1 계열), GRPO/RLVR, MoE·MLA 아키텍처                            | softmax 퍼셉트론 + RL | 다음 토큰 예측기 위에 사고 과정을 강화학습으로 형성                                      |

대학원생 과제 제안: 위 논문 중 하나를 골라 (1) 퍼셉트론·로지스틱 회귀의 어느 성질을 일반화했는지 (2) 실습 노트북 파트 1의 실험을 그 설정으로 확장할 수 있는지 발표

# 요약



## 구조

퍼셉트론 = 가중합  $z = w^T x$  + 비선형 활성화  $f$ . 노드·에지·가중치·바이어스( $x_0 = 1$ ).



## 동작

순전파  $\hat{y} = f(w^T x)$ . 활성화의 선택(sgn/step/ $\sigma$ /identity  $\rightarrow$  ReLU/GELU/SwiGLU)이 모델의 성격을 결정.



## 학습

오분류 시  $w \leftarrow w + y x$  (Rosenblatt) = 퍼셉트론 손실의 SGD.  $\sigma$ +CE, identity+MSE 모두  $\Delta w \propto \Sigma(y - \hat{y})x$ .



## 이론

분리 가능이면  $(R/\gamma)^2$  회 내 수렴. 로지스틱 GD는 최대 마진으로 수렴(암묵적 편향, 2018 $\rightarrow$ 2025 Muon). VC =  $d+1$ .



## 한계와 극복

AND·OR은 직선 하나로, XOR은 불가(1969). 은닉층 = 학습된 특징 변환으로 해결, 역전파로 학습(1986).



## 현재

트랜스포머 FFN = 2층 MLP(파라미터 2/3), 어텐션 = 입력 의존 가중합, LLM 출력 = softmax 퍼셉트론. KAN·SNN은 뉴런 정의를 재검토.

# 질문 및 정리

## Q & A

### 생각해 볼 질문

1. 학습률  $\eta$ 가 Rosenblatt 규칙의 수렴에 영향을 주지 않는 이유를 수식으로 설명하시오.
2. 은닉 뉴런이 1개뿐인 MLP는 XOR을 풀 수 있는가? 실험하고 이유를 설명하시오.
3. 트랜스포머 FFN의 폭을 4d에서 8d로 늘리면 파라미터·연산량은 어떻게 변하는가? 이것이 MoE의 동기와 어떻게 연결되는가?
4. 로지스틱 GD가 최대 마진으로 수렴한다면, 왜 실무에서는 여전히 가중치 감쇠(weight decay)를 쓰는가?

